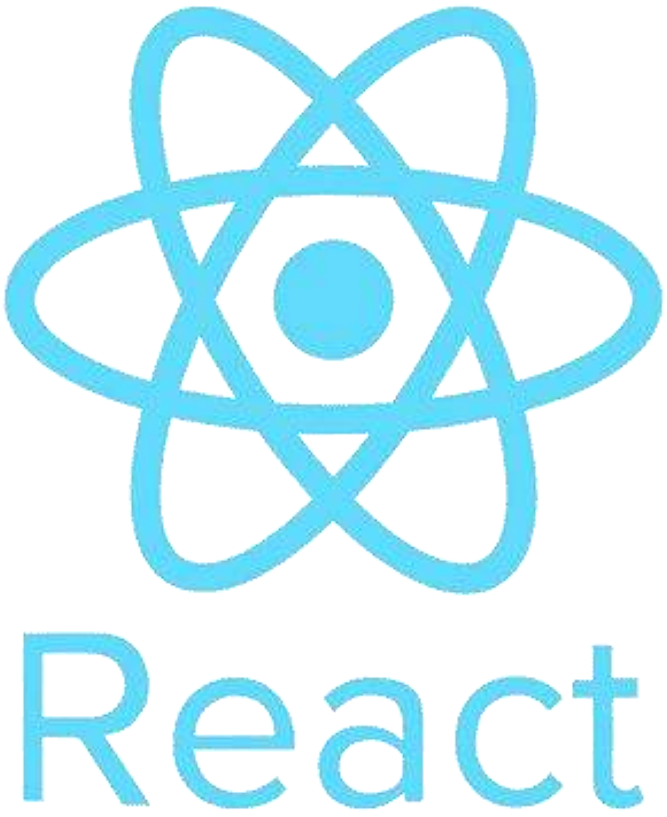
Welcome to React js



React.js

DAY-1

Topics Covered:

1. What is React.js?
2. History of React.js
3. Why do we need React.js?
4. Without React.js vs With React.js
5. Prerequisites for Learning React.js
6. Library vs Framework
7. SPA vs MPA (Single Page Applications vs Multi Page Applications)
8. Characteristics of React.js
9. Virtual DOM vs Real DOM
10. React.js Applications (Where React is Used?)
11. Installing React.js with Vite

Introduction to React js

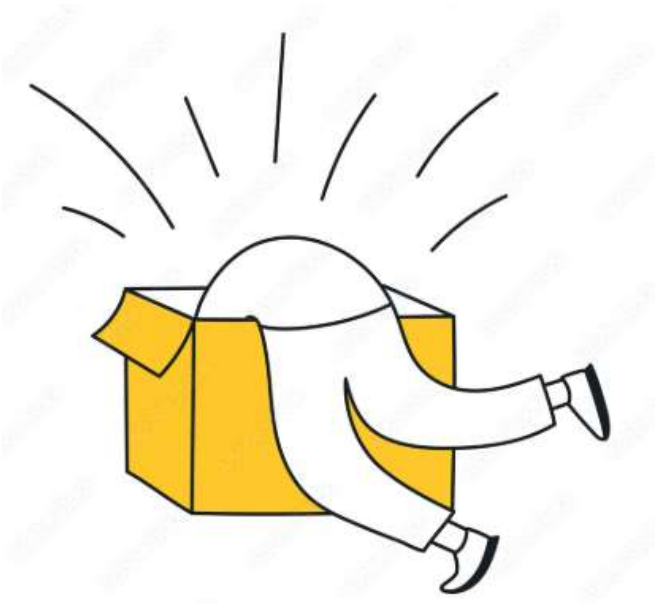
Q. What is React.js?

- React.js is a **Frontend Javascript library** used for building user interface (UI's).
- Specially react.js is used to create **single-page applications** (SPA).
- It allows developers to create reusable UI components, making the development process faster and more efficient.

History to React js

Q. Who created React.js?

- React.js developed by **Jordan Walke,** a software engineer at **Facebook.**
- He created react in 2011 as internal tool to improve the performance of Facebook's **News Feed** Section.
- React was officially **released to the public in 2013** as an open-source JavaScript library.
- It is now an **open-source** library used by developers worldwide.
- As of March 2025, React.js has had a total of **19 major versions** released.



Why do we need React js

Q. Why react.js?

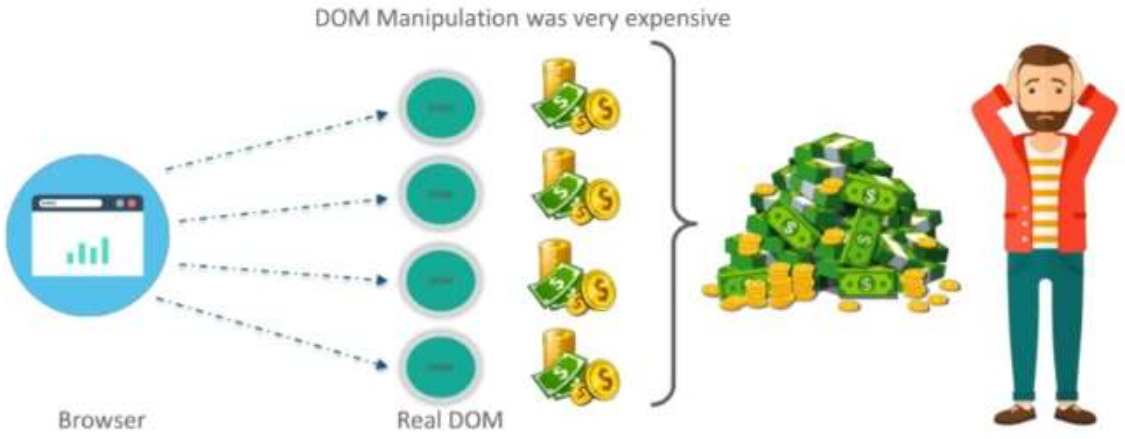
- Before React, developers used **vanilla JavaScript** and **jQuery** to build dynamic web applications.
- While working on this, they had limitations that made development slow and difficult to maintain.
- React was created to solve these problems and improve web development efficiency.
- **Problems with Traditional JavaScript/jQuery**

1. Complex DOM manipulation
2. Slow Performance due to Real DOM updates

Without React.js vs With React.js

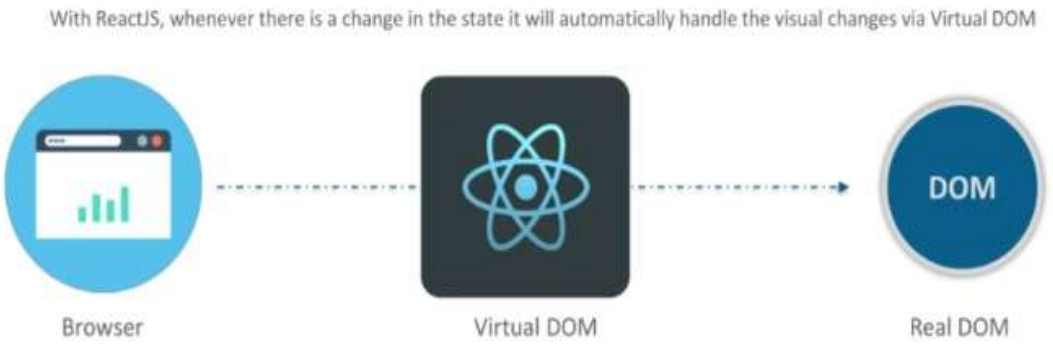
Q. Without react.js?

- In **traditional JavaScript**, every UI update directly modifies the **Real DOM**. The problem is that the DOM is **slow.**
- And DOM manipulation was very expensive.



Q. With react.js?

- React does not update the **Real DOM** directly.
- Instead of it uses **virtual dom**.
- Keeps UI fast and smooth, even in large applications.



Prerequisites for Learning React.js

HTML, CSS, JAVASCRIPT:

i. **HTML (Hyper Text Markup Language)** – The structure of a webpage (e.g., headings, buttons, forms).

ii. **CSS (Cascading Style Sheets)** – Used for styling the webpage (e.g., colors, layouts).

iii. **JavaScript (ES6+)** – Arrow functions, let and const variables, template literals, destructuring (arrays & objects), spread and rest, modules (import/export), events, and event handling.

Library vs Framework

Q. What is Library?

- A **library** is a collection of reusable functions or components that developers can use as needed.
- React is a **UI library** that helps build components but does not dictate how the whole application should be structured.

Q. What is Framework?

- A **framework** is a full-fledged system that provides rules and structure for building an entire application.
- It controls the architecture, routing, and state management (e.g., Angular, Vue.js).

SPA vs MPA

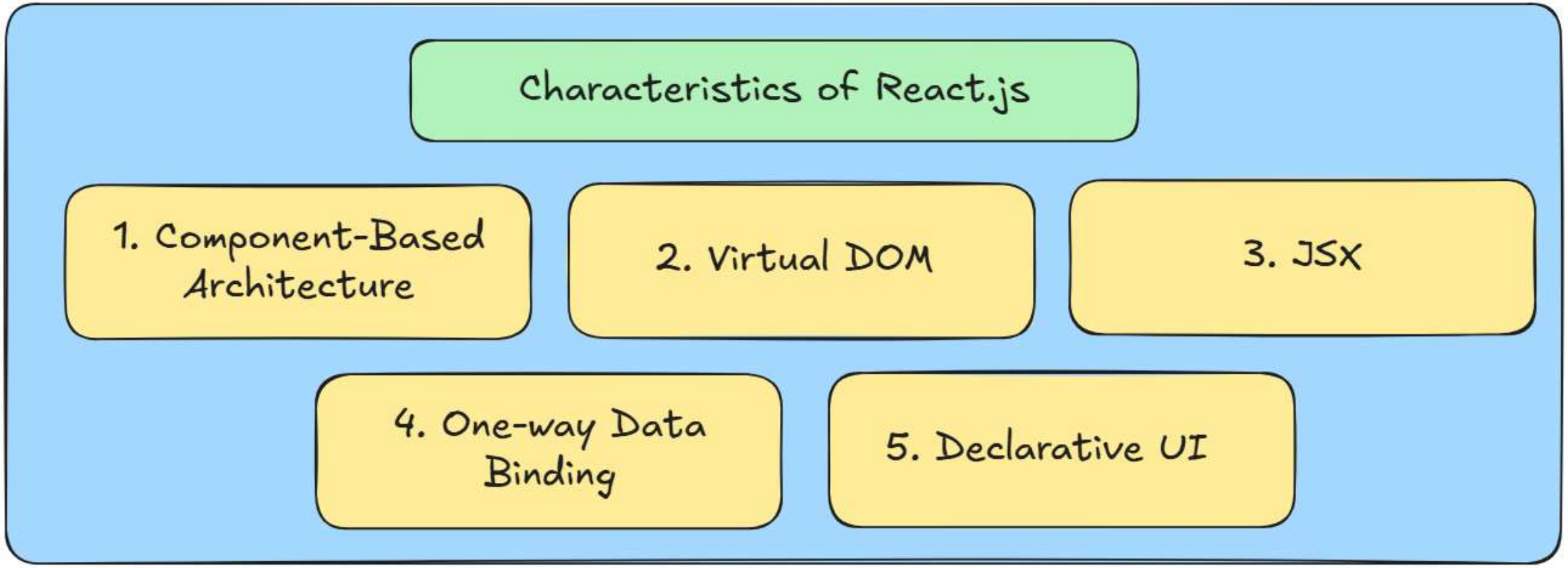
Q. What is SPA?

- A SPA stands for **single-page application**.
- Loads a **single HTML page** and updates content dynamically without reloading the page.
- **Faster after the first load**, as only necessary data is updated.
- Uses **client-side routing** (React Router) to switch views without reloading.
- Examples: **Gmail, Facebook, Twitter, Netflix, YouTube Web App**.

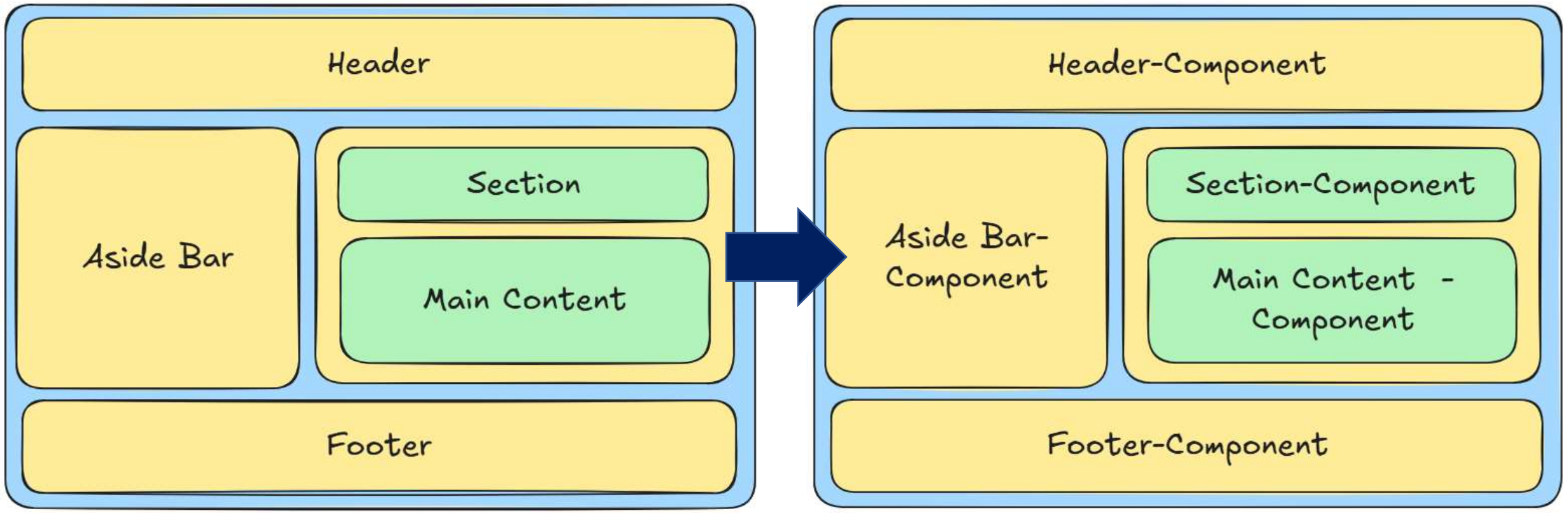
Q. What is MPA?

- A MPA stands for **Multi-page applications**.
- Each page is a **separate HTML file**, making navigation slower.
- **Slower performance**, as the browser loads everything again on each page change.
- Every new page request **reloads the entire page from the server**.
- Uses **server-side routing** (URLs trigger full-page reloads).
- Examples: **Amazon, Wikipedia, News Websites, Banking Portals**.

Characteristics of React js

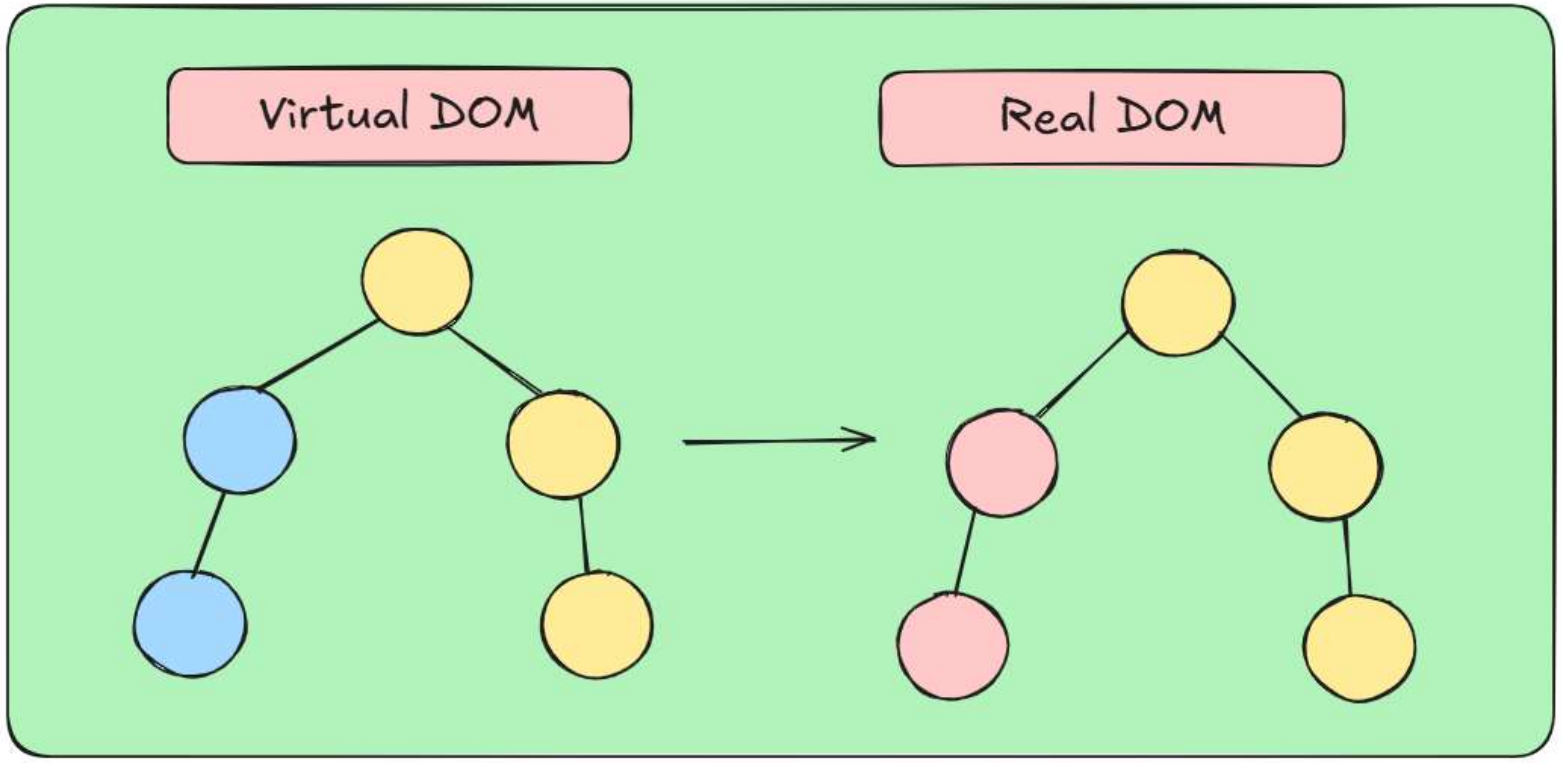


1. Component-Based Architecture



2. Virtual DOM

- React uses a **Virtual DOM** to update only the changed parts of the UI.
- Improves speed by avoiding direct manipulation of the Real DOM.
- Example: Updating a single item in a list does not reload the entire page.



3. JSX

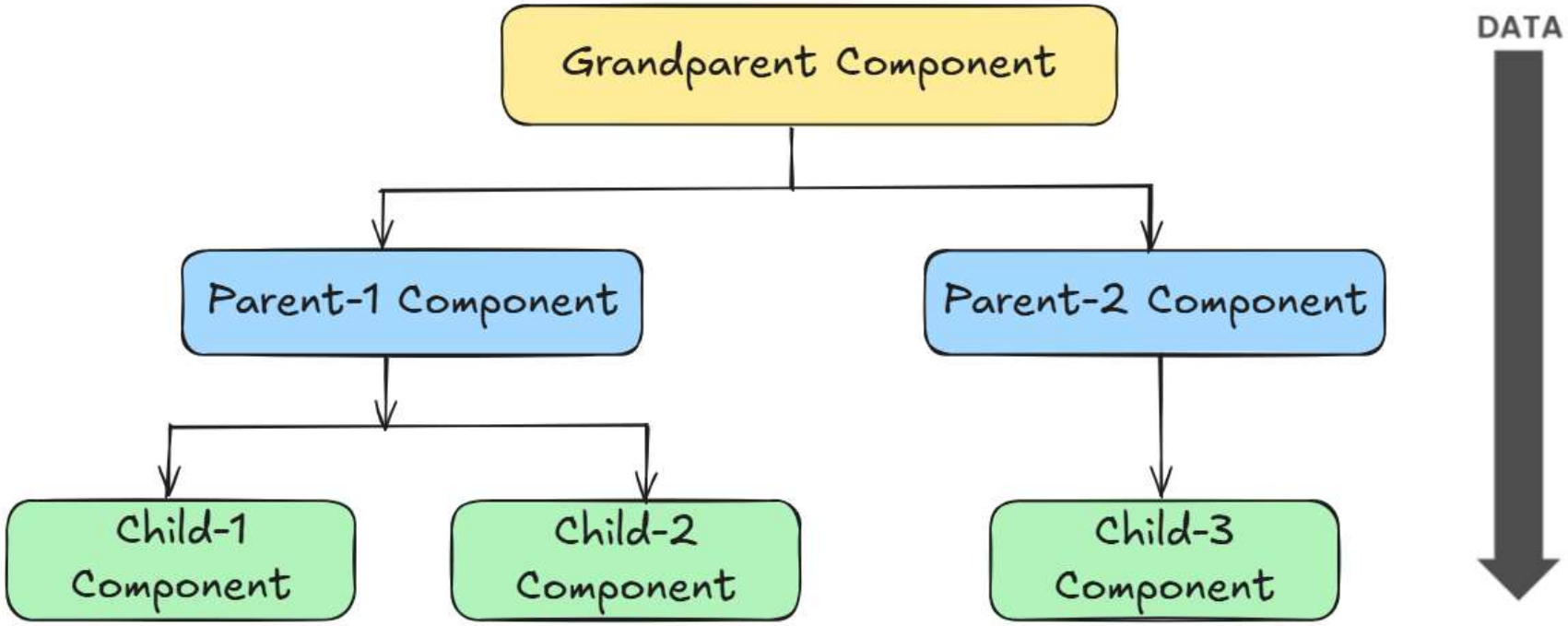
- JSX stands for **Javascript and XML**.
- JSX allows writing **HTML-like syntax inside JavaScript**.
- Makes UI development **more readable and declarative**.

Example:

```
const element = <h1>Hello, React!</h1>;
```

4. One Way Data Binding

- Data flows in **one direction (from parent to child components)**.
- Example: A **parent component** passes data to a **child component** via props.

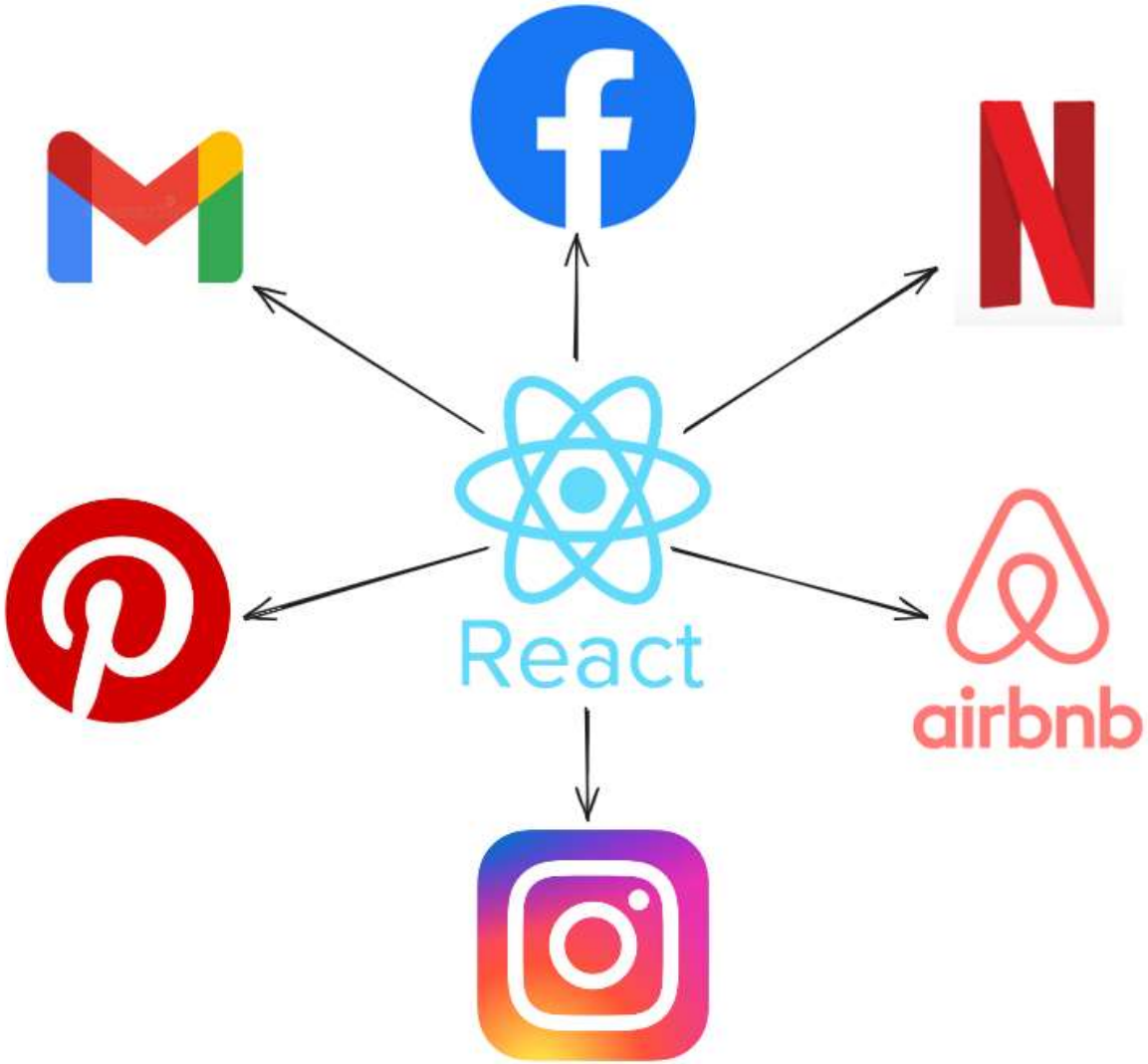


5. Declarative UI

- React **automatically updates the UI** when data (state) changes.
- Developers only describe **what** the UI should look like, and React handles the rest.

Imperative	Declarative
How to do things.	What to do.
Statements.	Expressions.
Defining variables and changing their values.	Evaluate result based on input.

React js Applications

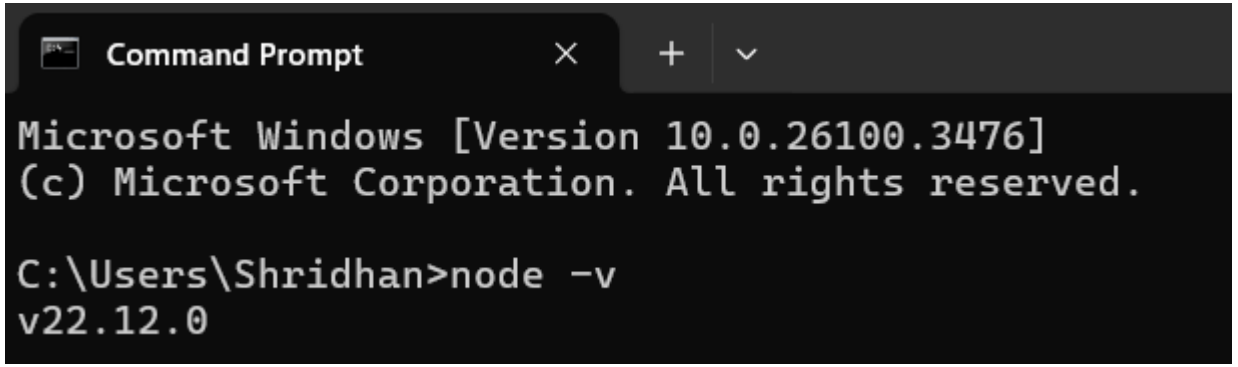


Installing React.js with Vite

- **Vite** is frontend build tool and development server.
- Vite is a **faster and lightweight**.
- It provides **better performance and a smoother developer experience**.

Step-1: Install Node.js

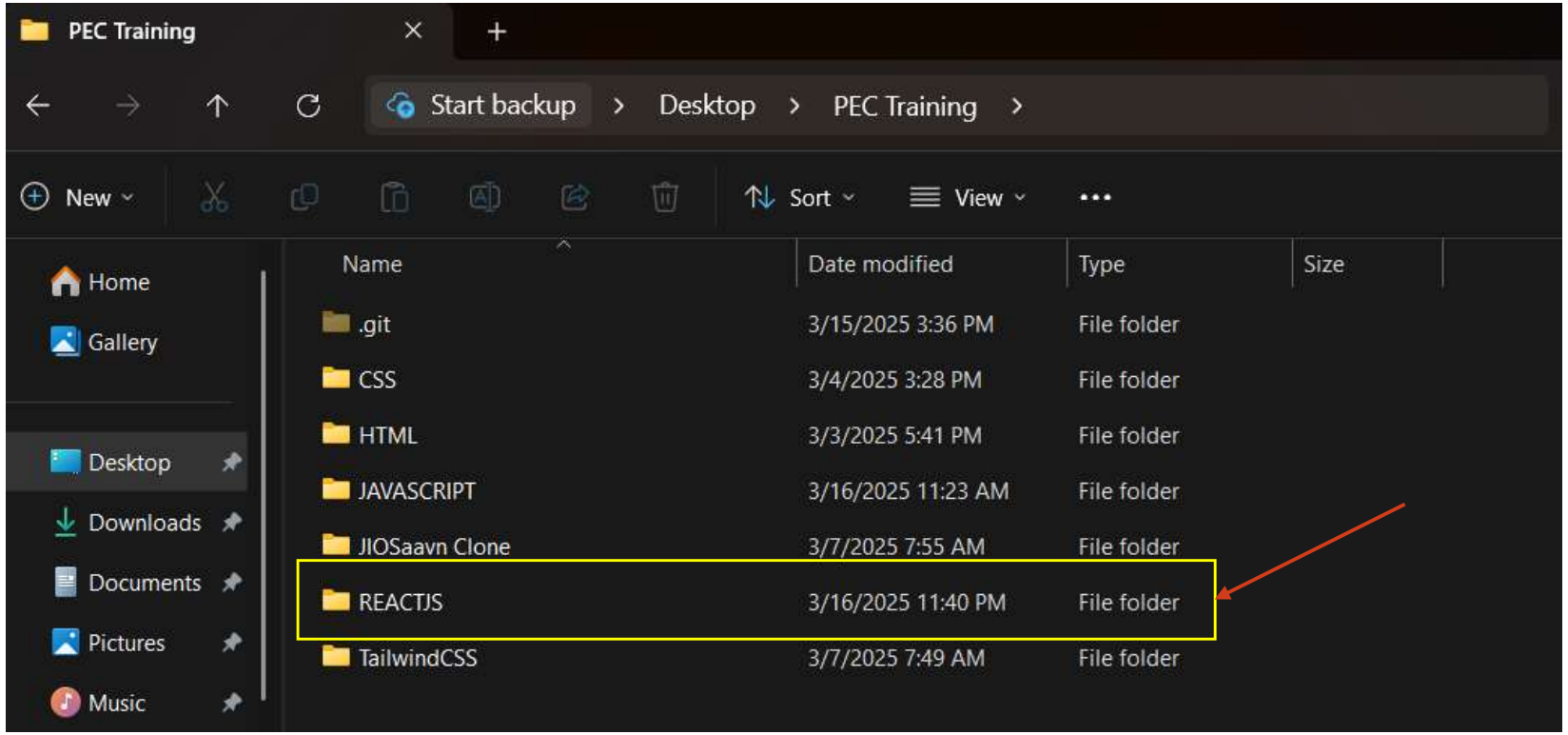
- Before installing React, make sure **Node.js** is installed on your system.
- After installation open **command prompt** (cmd) to check the version of installed node .js
- Command: **node -v**



☞ **If not installed, download it from: https://nodejs.org/**

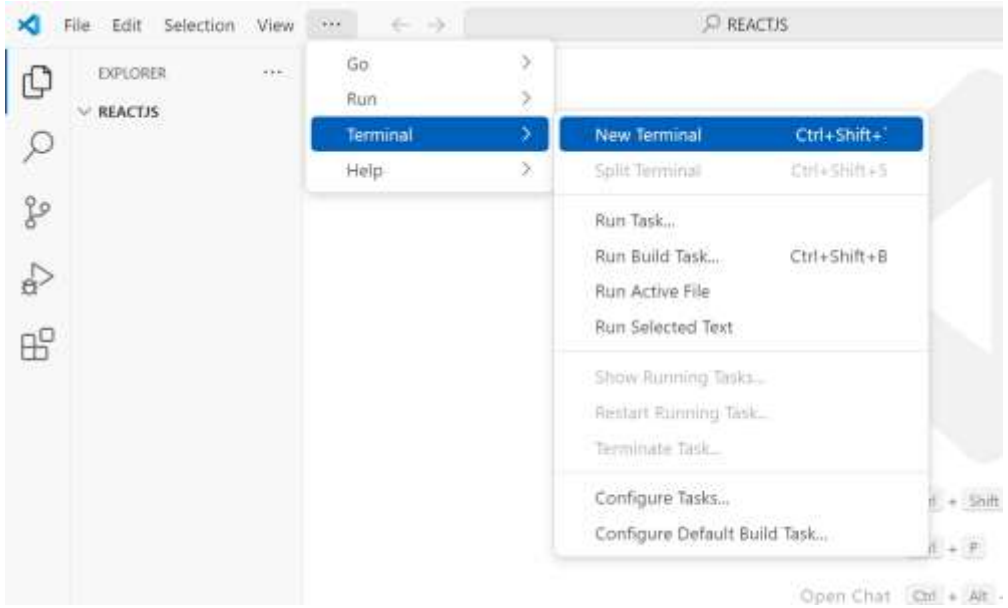
Step-2: Create a New Folder (name-Reactjs)

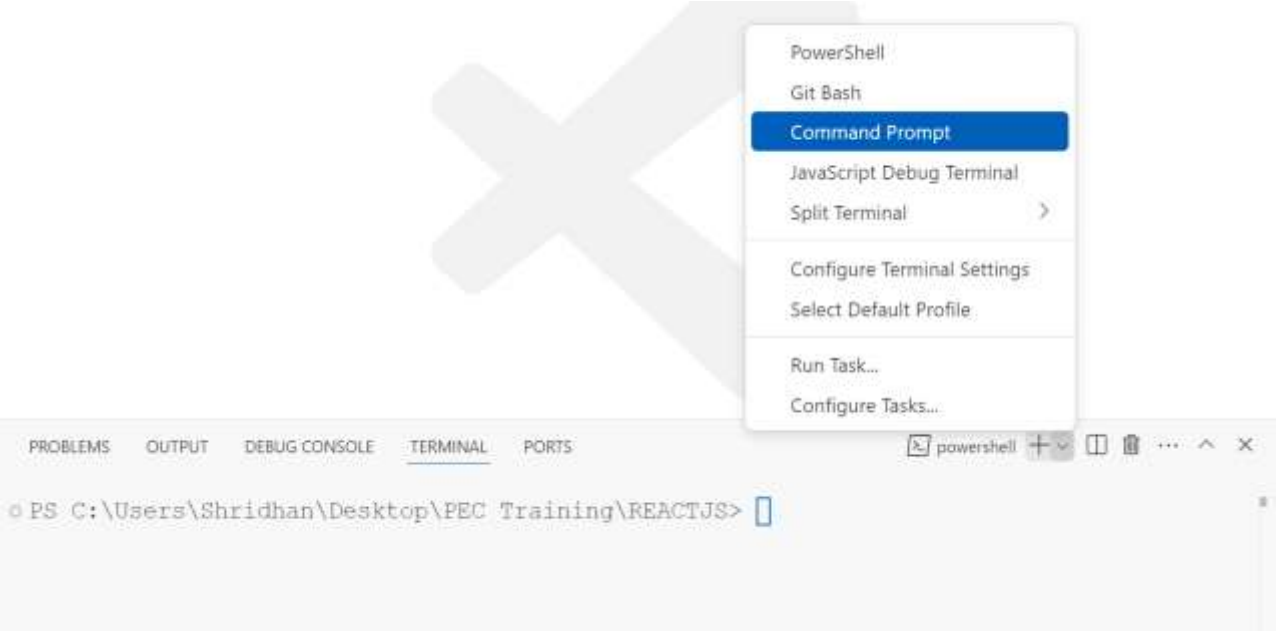
- Create a new folder inside web tech program training folder.
- Name of the folder - Reactjs



Step-3: Open this folder inside the vs code

- Click Terminal – New Terminal
- Click (+) icon and open the **command prompt** not Powershell.





Step-4: Run the following command in cmd

- Command: **npm create vite@latest**
- And just enter on keyboard.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shridhan\Desktop\PEC Training\REACTJS>npm create vite@latest
```

powershell
cmd

Step-5: Packages and Project Name

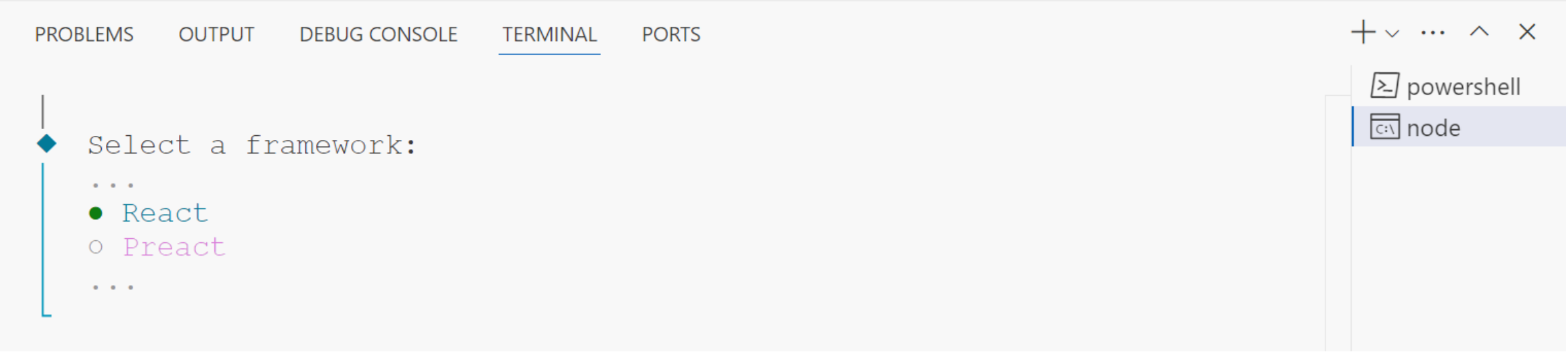
- It will ask to install some packages, you have to give the permission by typing:
- **Ok to proceed: y**
- Then after hitting enter it will ask for project name.
- Give the project name whatever you want.
- For Example - **Project name**: **my-first-create-app**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

> npx
> create-vite

◆  Project name:
   my-first-create-app
```

powershell
node

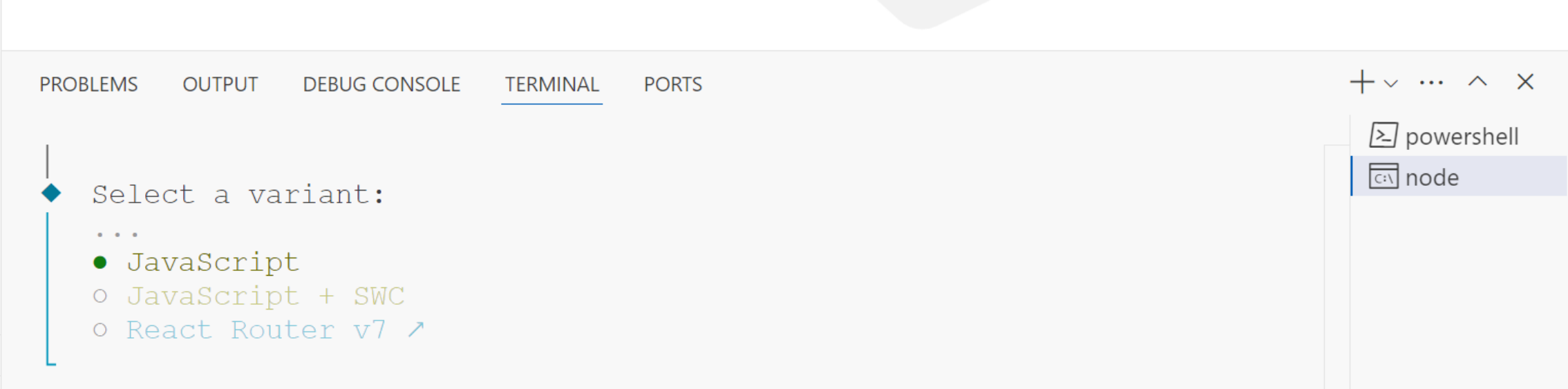
Step-5: Select a framework

- After giving name it will ask the select a framework.
- Only Select – **React**
- How to select ? – Use down arrow key on keyboard for selecting option react option.
- For Example – **Select a framework: React**



Step-5: Select a variant

- After selecting a framework it will ask for you variant.
- Variant is nothing but which language you are going to use in react js
- How to select ? – Use down arrow key on keyboard for selecting option **javascript** option.
- For Example – **Select a variant: Javascript (✓)**
- **Don't select Javascript+swc (✗)**



Step-6: 3 commands will be created

- After that it will create one folder inside the main react js folder.
- You will able to see the 3 commands in the terminal.
- **Now use step-7 on next slide**

Step-7: Use first command

- Copy the **first command** and paste in cmd.
- After that hit enter on the keyboard.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
└  Done. Now run:

  cd my-first-create-app
  npm install
  npm run dev


C:\Users\Shridhan\Desktop\PEC Training\REACTJS>cd my-first-create-app
```

powershell
cmd

Step-8: Use second command

- Now you are entered in the react project folder.
- Copy the second command and paste into the cmd.
- And hit enter, it will install important packages which will help to run the react project.
- Command : **npm install** (It will create one node-modules folder)
- It will take some tine to install the packages. so wait for some time. (It might takes some minutes also, it depends upon your internet speed)

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
└ Done. Now run:

  cd my-first-create-app
  npm install
  npm run dev


C:\Users\Shridhan\Desktop\PEC Training\REACTJS>cd my-first-create-app

C:\Users\Shridhan\Desktop\PEC Training\REACTJS\my-first-create-app>npm install
```

After Installation

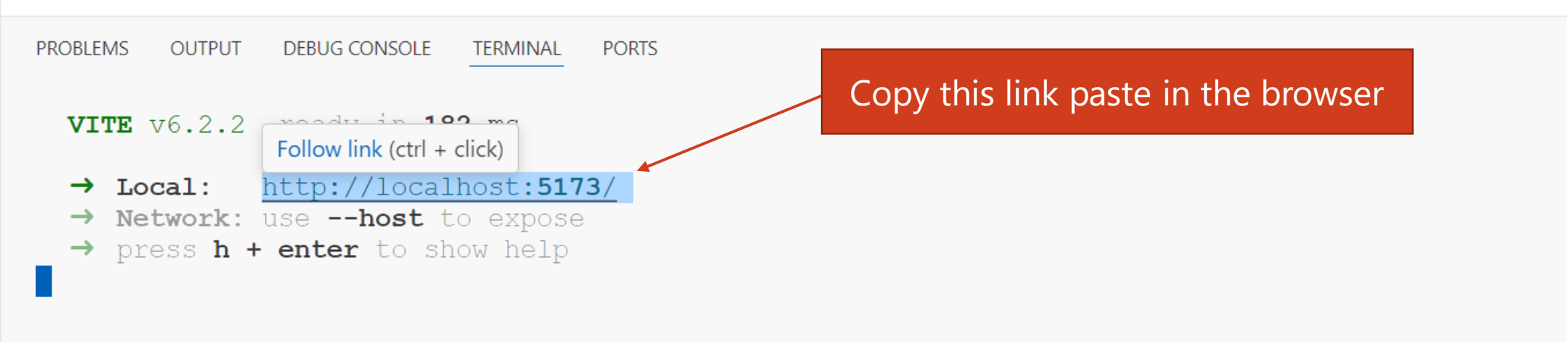
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS


C:\Users\Shridhan\Desktop\PEC Training\REACTJS>cd my-first-create-app

C:\Users\Shridhan\Desktop\PEC Training\REACTJS\my-first-create-app>npm install

added 149 packages, and audited 150 packages in 22s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Step-9: Use third and last command

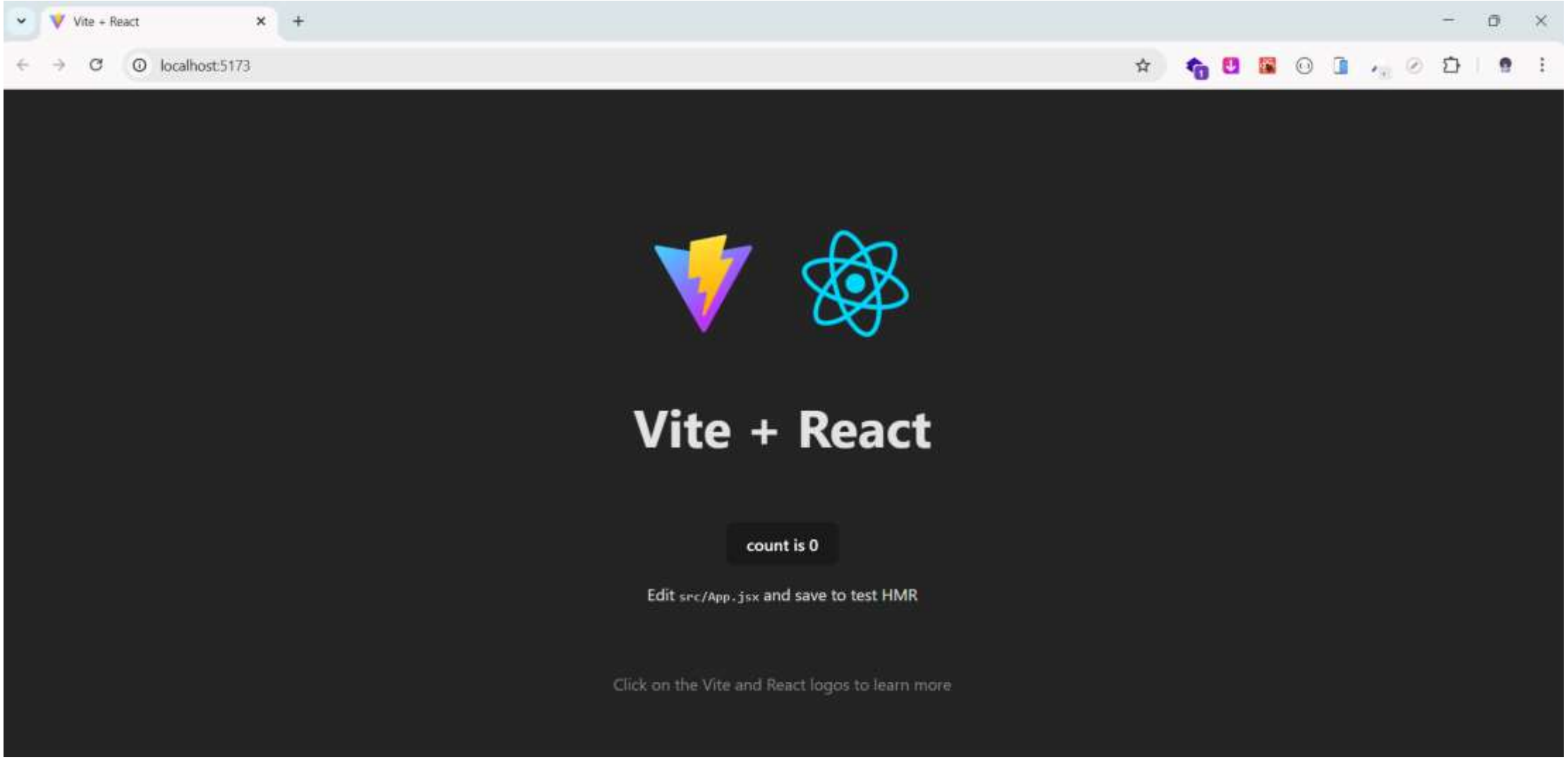
- Now you can right side image, your folder structure will look like this.
- That means you have created your first react app.
- To run react project you need last command copy the command and hit enter.
- Command: **npm run dev**

Step-10: Copy the link and Paste into browser

- After the **step-9** it will create one link.
- Just copy the link and paste it into the browser.
- Now you can see react project is running on the browser successfully.



Congratulations you have created first react project



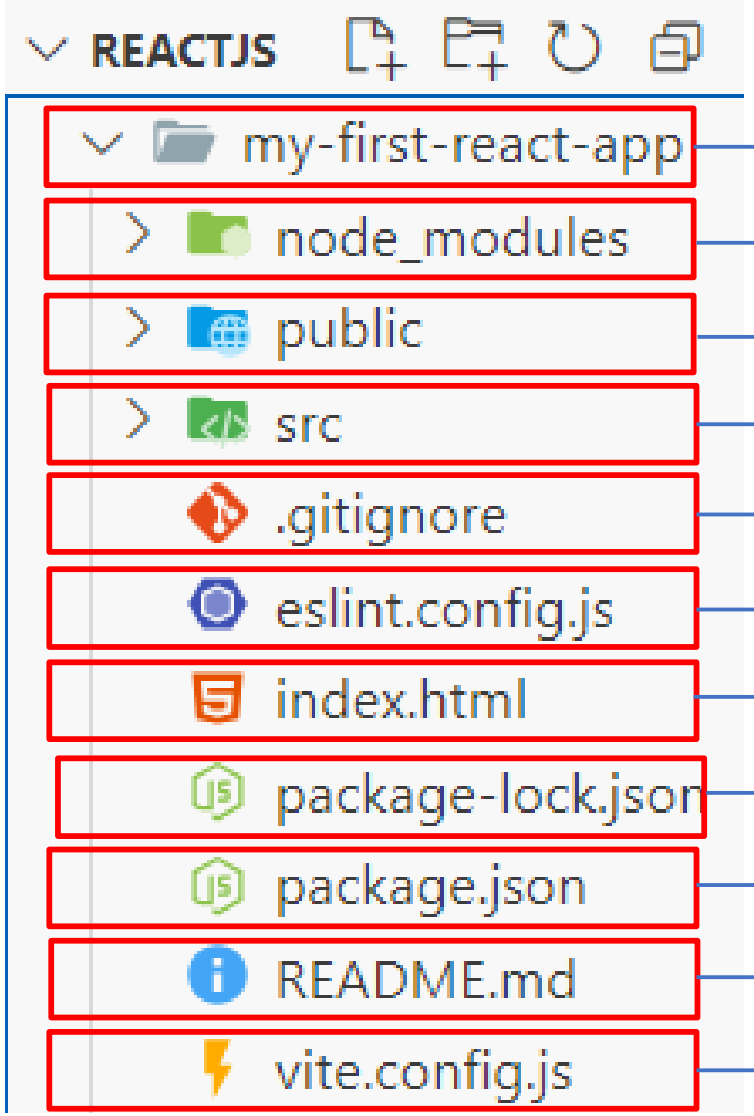
React.js

DAY-2

Topics Covered:

1. React Folder Structure
2. React Folder Setup
3. JSX and JSX Rules
4. Fragments
5. JSX Expression
6. Components
7. List and Keys
8. Props

React Folder Structure



- my-first-react-app → Main React folder
- node_modules → Stores all installed dependencies (auto-generated).
- public → Contains static data likes images, and icons.
- src → Main Source code folder (Components, styles, and logic)
- .gitignore → Specifies files Git should be ignored. (e.g. node_modules)
- eslint.config.js → Configuration of ESLint (Code linting and formatting)
- index.html → The main HTML file where React attaches
- package-lock.json → Auto-generated file to lock dependency versions.
- package.json → Defines project dependencies, scripts, and metadata.
- README.md → Documentation for the project (how to use/setup).
- vite.config.js → Configuration file for Vite (fast build tool for React).

React Folder Setup

- Open the project in vs code
- Delete the **src** folder to remove any pre-provided jsx and js files
- Create the src folder. This folder belongs to react.
- Whatever code we have for react we need to write in **src** only with **.jsx** extension
- Inside **src**, create **main.jsx** and **App.jsx**
- To create react application, we need two important modules

1. react
2. react-dom/client

main.jsx

my-first-react-app > src > main.jsx

```jsx
import React from "react";
import ReactDOM from "react-dom/client";

ReactDOM.createRoot(document.getElementById("root")).render(
  <h1>Hello React</h1>
);
```

JSX (Javascript and XML)

- JSX stands for **Javascript and XML**.
- It is a syntax extension for JavaScript that looks like HTML but works inside JavaScript.
- This syntax allow us to write html code in react.
- It looks similar to html but it is not.
- Without jsx it is very difficult to create react application.

Q. Why do we need JSX?

1. Easier to write and read UI components
2. Allows us to write down mix code of javascript logic with UI components

Rules of JSX:

1. JSX must have **one parent element**
2. Self-Closing tags must have a **/**
3. Use **{}** for Javascript expressions
4. JSX attributes use **camelcase** convention
5. To provide class in react we have to use **className** as **class** is a keyword in javascript.
6. To link your label with input tag, we have to use for as **htmlFor** as **for** is a keyword in javascript.
7. User defined custom components should be in **PascalCase**.

What is Fragment in React.js?

- A fragment in react is used to **group multiple elements** without adding an extra HTML element like a ** .**
- Fragment is not a html element, it is react component.
- We **can not pass id** or any other attribute to it. It is not adding to the DOM, then how can we pass any attribute to it.
- It accepts only one attribute that is **key**, we will see this key later.
- If we want to provide attributes, we need to go with html elements.

Q. How to use Fragments?

1. Using **<React.Fragment>**

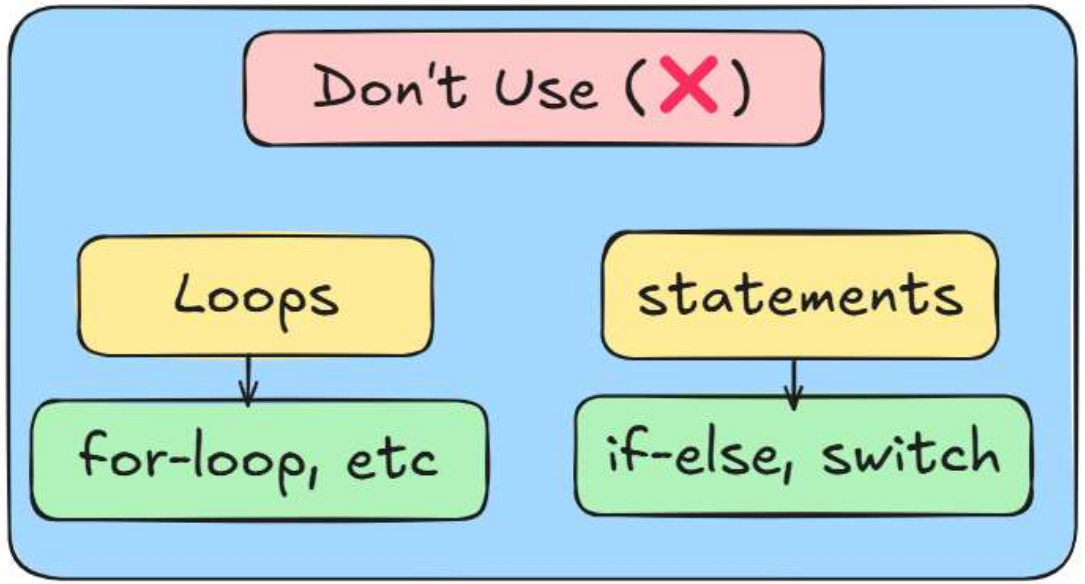
2. Using Short sugar Syntax (**<>** and **</>**)

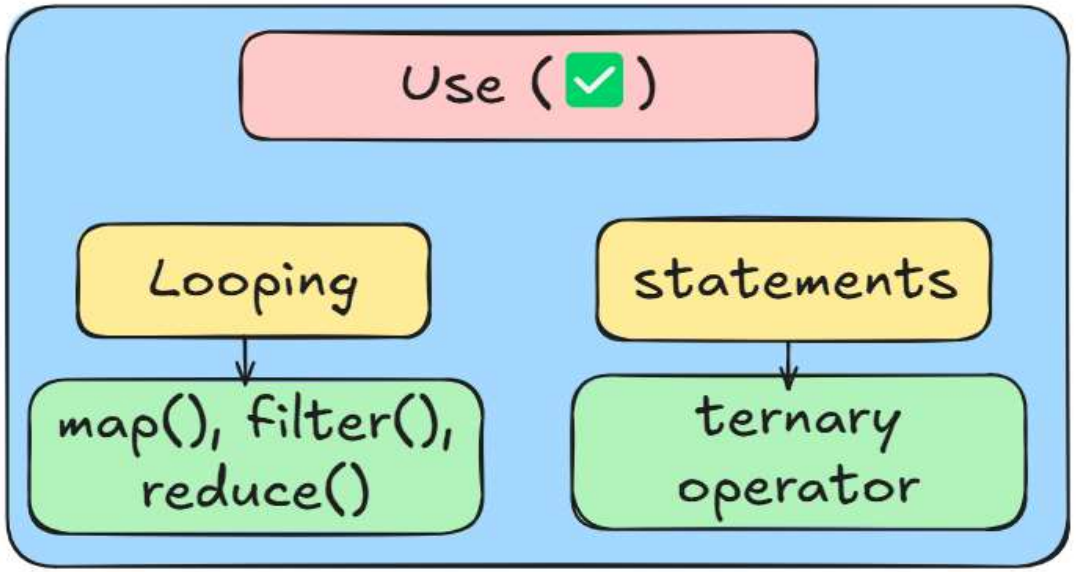
Suggestion: A shorter way to use fragments is by using empty angle brackets. (**<>** and **</>**)

Note:We cannot pass key attributes in short fragment syntax.

JSX Expression

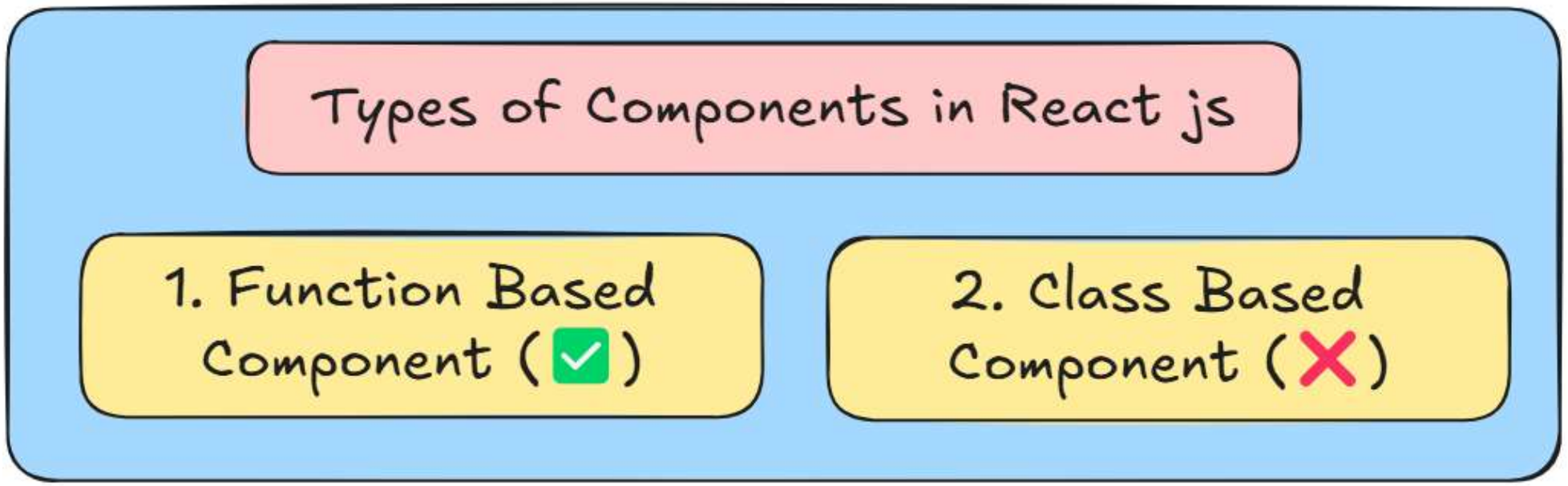
- To use js variables or js syntax in our jsx template, we have to use a pair of curly braces, this is called a **jsx expression { }**.
- Don't take it as an object, a pair of curly braces in the jsx is a jsx expression.
- Important Note: Inside expression we cannot render boolean/null/undefined but we can use with conditions.



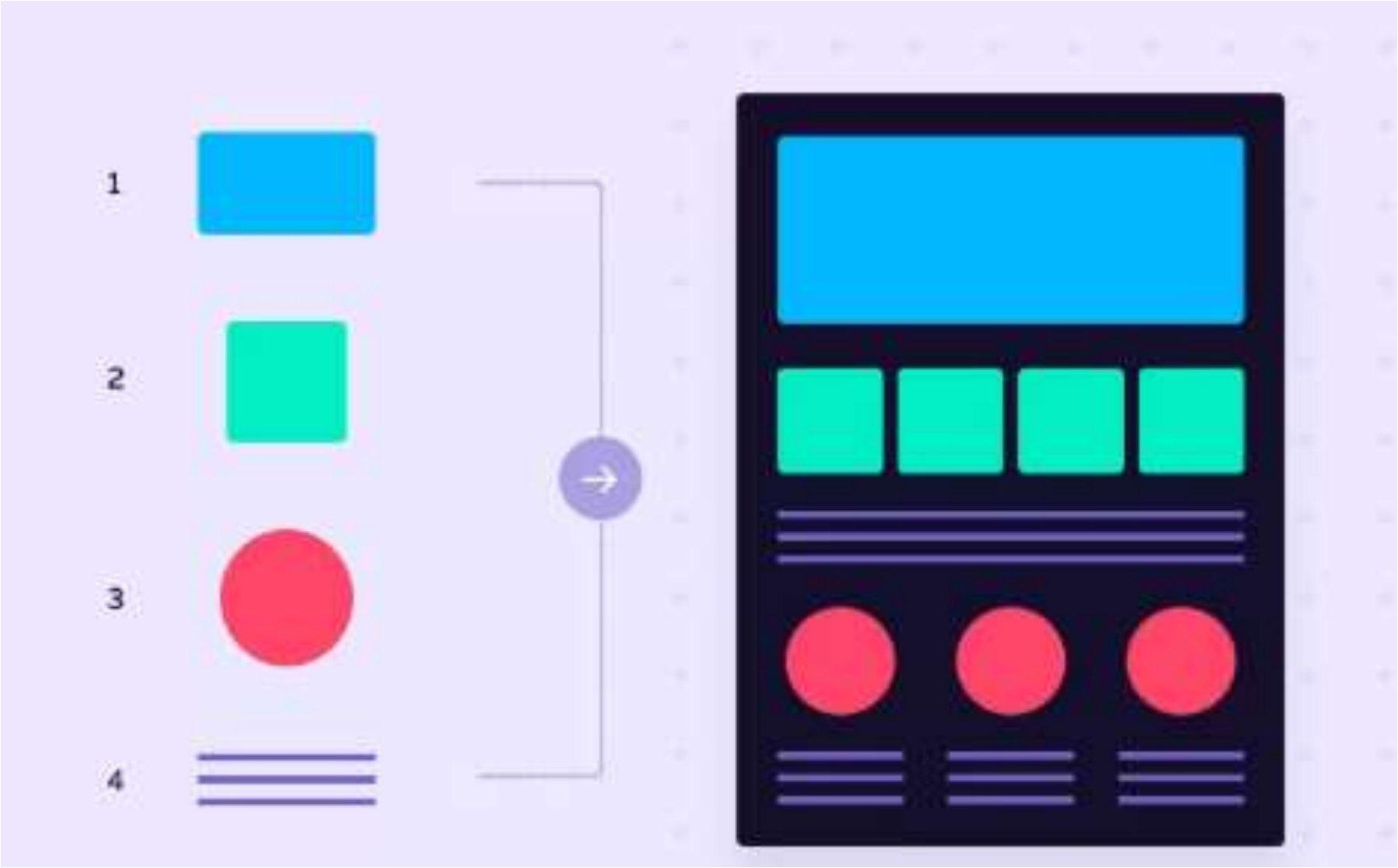


Components in React js

- React follows component-based architecture.
- A component in react is **reusable**, **independent** javascript functions that can be used multiple times within an application.
- Components allows us to **break down** the UI into smaller parts.



Components in React js



How to create component?

- Create a new component file in the **src** folder - You should use the extension as **.jsx** only.
- The first alphabet of the file name should be capital Eg - **App.jsx**
- You can use the syntax of either class-based-component or function-based-component.
- Then export your component.
- And import it with proper address wherever you feel necessary and use it as custom defined tag (wrap inside angular braces)

Function Based Component in React js

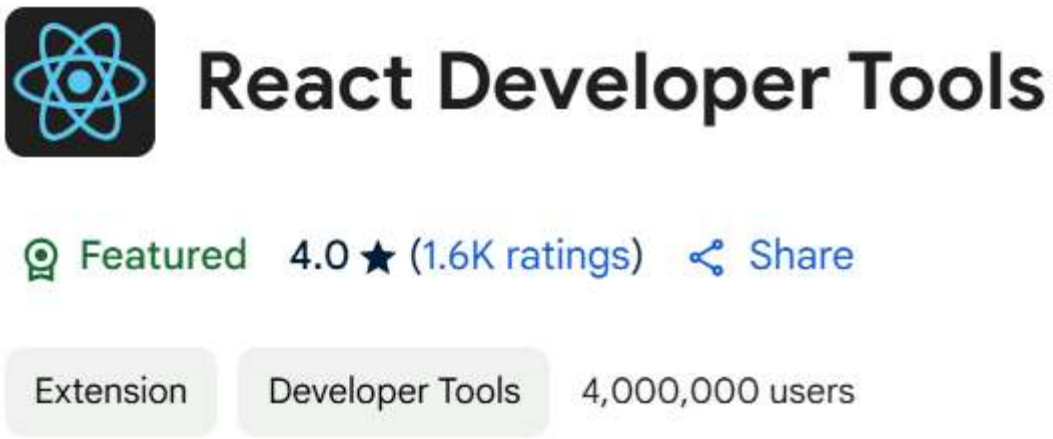
- A functional component is just a Javascript function that returns JSX.

Syntax of Function Based Component

```
let ComponentName = () => {
  return JSX;
};

export default ComponentName;
```

Extension – React Developer Tools

- **React Developer Tools** is a browser extension that helps inspect and debug React components in real-time.
- It allows developers to view component hierarchy, props, state, and performance, making debugging easier.



List and Keys in React js

Q. What are Lists in React?

- In React, a list is a **collection of items** (like an array) that we render dynamically using **.map()**

Q. Why Do We Need Lists?

1. Helps display multiple items dynamically.
2. Makes UI updates efficient.
3. Useful for rendering data fetched from an API.

List and Keys in React js

Q. What are Keys in React?

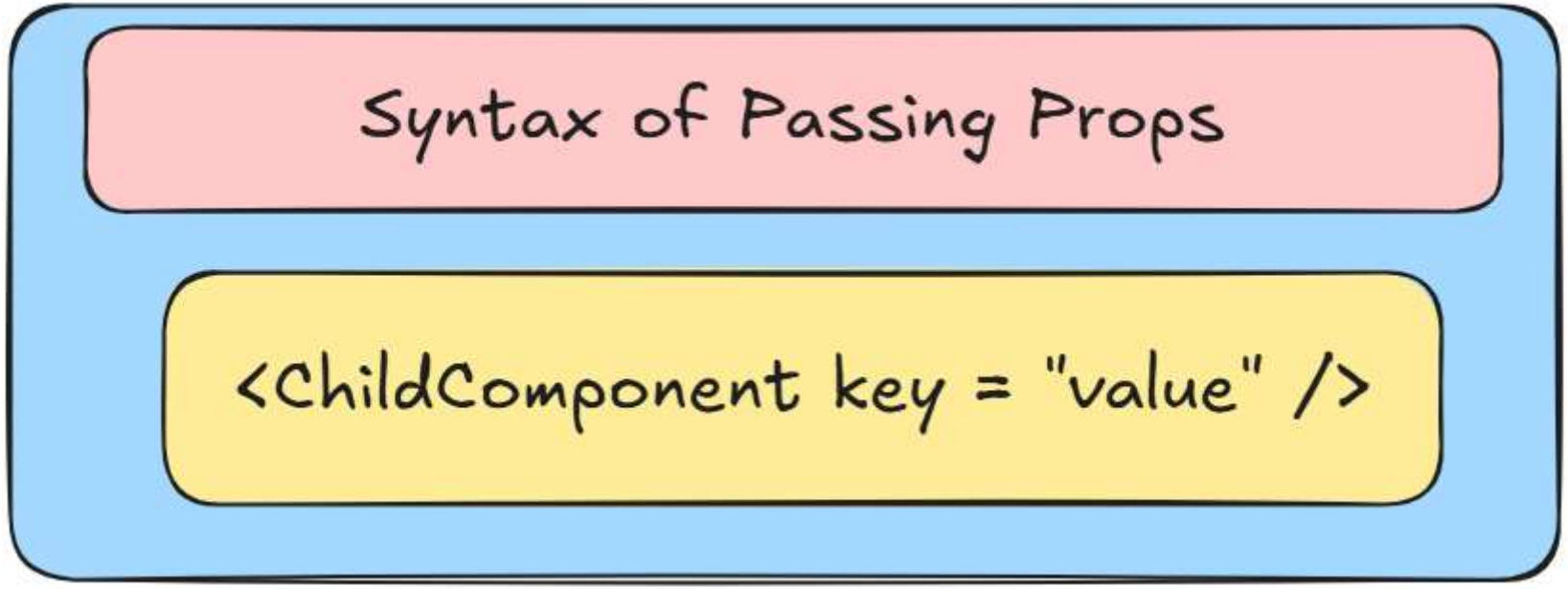
- In React, **Keys** are special attributes that help React identify which items changed, added, or removed.
- **Syntax:** items.map((ele, index) => <Component key={index} />);
- **Important Note: A key should be unique for each list item.**

Q. Why Do We Need Keys?

1. Improves performance by helping React update only the changed items.
2. Prevents unnecessary re-renders.
3. Helps React track elements efficiently in a list.

Props in React js

- Props stands for **properties** in react js.
- Props are used to pass data from a parent component to a child component in react. (**Unidirectional Flow**)
- Props are **read-only** and help make components reusable and dynamic in nature.
- We can pass any javascript value through props and it is send as an object.



How to pass and consume props?

Passing Props

```
1  import React from "react";
2  import ChildComponent from "./ChildComponent";
3
4  const ParentComponent = () => {
5    return (
6      <div>
7        <h1>ParentComponent</h1>
8        <ChildComponent name="john" />
9        <ChildComponent name="bob" />
10       <ChildComponent name="dave" />
11     </div>
12   );
13 };
14
15 export default ParentComponent;
```

Consuming Props

```
1  import React from "react";
2
3  const ChildComponent = (props) => {
4    return (
5      <div>
6        <h1>ChildComponent</h1>
7        <h1>{props.name}</h1>
8      </div>
9    );
10 };
11
12 export default ChildComponent;
```

How to pass different javascript values as a prop

- If you are passing string you can just pass it normally in quotes (“ ”), we can not use backticks.
- If you want to send any data type other than string, we should go with expression ({ }).
- We need to wrap data inside jsx expression ({ }).
- We can also pass string in jsx expression.

```jsx
<ChildComponent
        name="john"
        salary={50000}
        isAvailable={true}
        isAlive={undefined}
        doesExist={null}
        skills = {["HTML", "CSS", "JS"]}
      />
```

```jsx
const ParentComponent = () => {
  const person = {
    name: "John",
    salary: 50000,
    isAvailable: true,
    isAlive: undefined,
    doesExist: null,
    skills: ["HTML", "CSS", "JS"],
  };
  return (
    <div>
      <h1>ParentComponent</h1>
      <ChildComponent person={person} />
    </div>
  );
};
```

Props Children

- Till now we are using unpaired syntax for our custom tags, but what if we use the paired way and pass some content in between.
- React automatically will convert that content into props and it will be present inside your props object in the children key.
- This is called as **props children**.
- You can access it through props.children in child component.
- We can pass JSX only as content in props.children (between opening and closing custom tags).
- The passed JSX is present in the form of array in the children key.

Props Children

```
<ChildComponent>
        <h1>Hello, John</h1>
        <p>This is passed as children to props</p>
</ChildComponent>
```

```
<div>
        <h1>ChildComponent</h1>
        <div>{props.children}</div>
</div>
```

```
props
  children: [<h1 />, <p />]
    0: <h1 />
    1: <p />
    new entry
  new entry: ""
```

React.js

DAY-3

Topics Covered:

1. Events in React
2. Event Handling in React
3. States
4. Conditional Rendering
5. Styling in React

Events in React

- Events in React are user interactions such as clicks, key presses, or mouse movements.
- React handles events using event listeners, similar to JavaScript, but with a slightly different syntax.

Important Points:

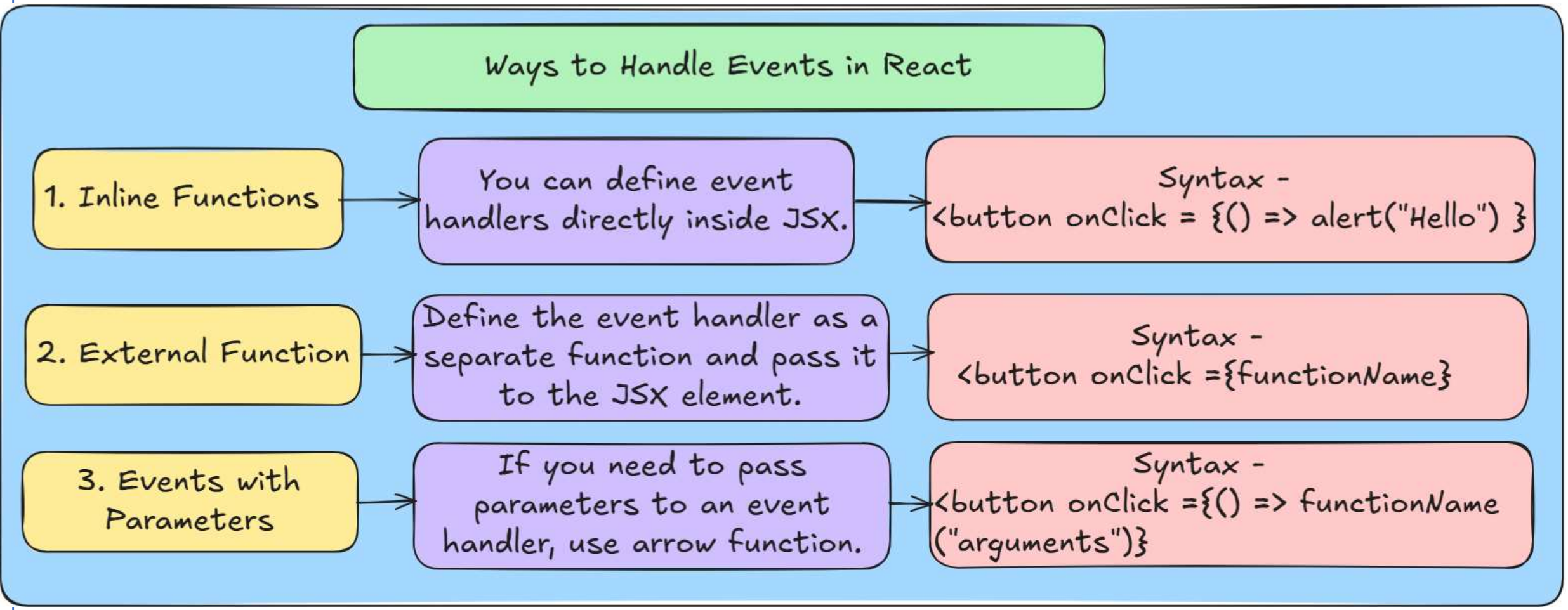
1. React events are named using **camelcase** (e.g. onClick, onChange)
2. Event handlers are functions that execute when an event occurs.
3. The event object (**e**) provides the information about the event.
4. The **target** property in the event object refers to the element that triggered the event.

   Syntax – **e.target**

Events Handling in React

- In react, event handling is similar to handing events in javascript but with some key differences.
- React uses a **SyntheticEvent**, a wrapper around native events for cross-browser consistency.

Events Handling in React



e.preventDefault() in React

- In react, **e.preventDefault()** is used to **prevent the default behaviour** of an event.
- This is commonly used in **submissions forms**, **links clicks**, and other default browser behaviors.
- Syntax – **e.preventDeafult()**;

Q. Why Use e.preventDefault()?

By default, some HTML elements have predefined behaviors:

1. **Forms (<form>)** – Refresh the page on submission
2. **Links (<a>)** – navigate to another page

States in React

- State in React is a special kind of variable that stores data in a component and makes React to re-render the UI whenever it changes.
- **useState** – It is a hook in react js which will handle state management in function based component.

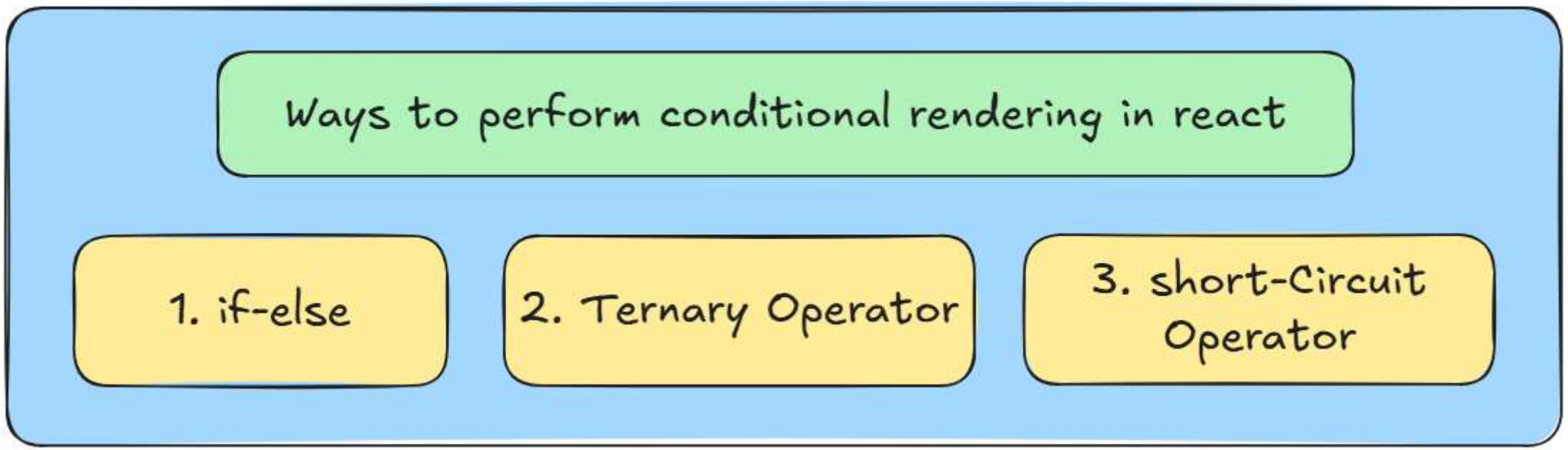
Syntax

```
const [state, setState] = useState(initialValue);
```

- **state** – Holds the current value or initial value
- **setState** – It is a function which will update the current value

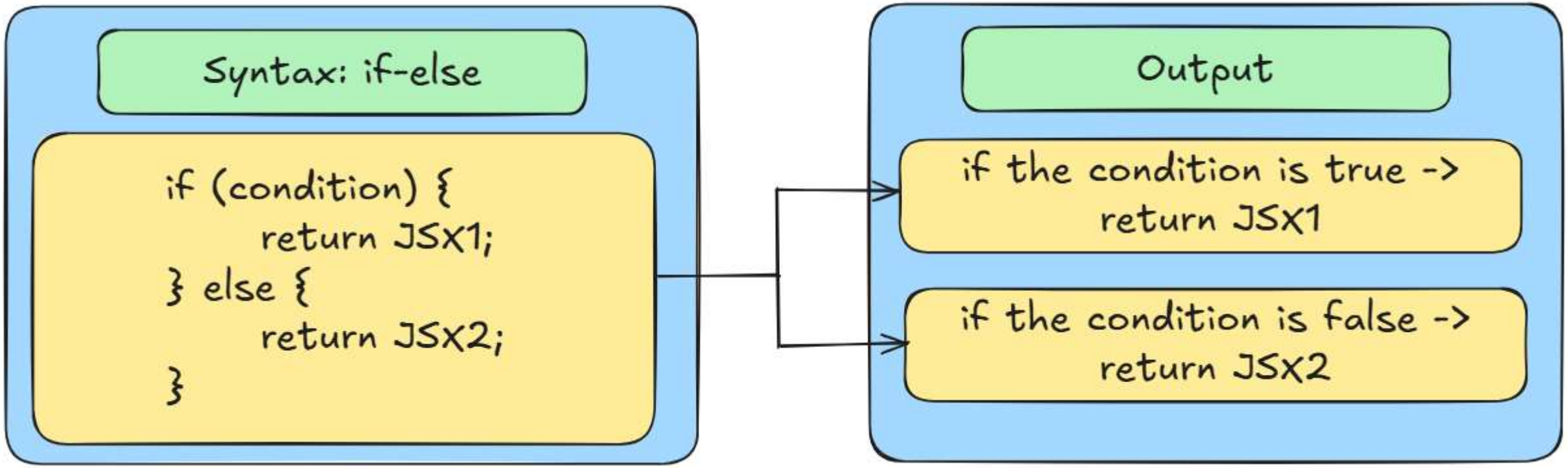
Conditional Rendering in React

- Conditional Rendering, in react means **showing different UI elements based on a condition**.
- Instead of rendering everything at once, React **dynamically** decides what to render.



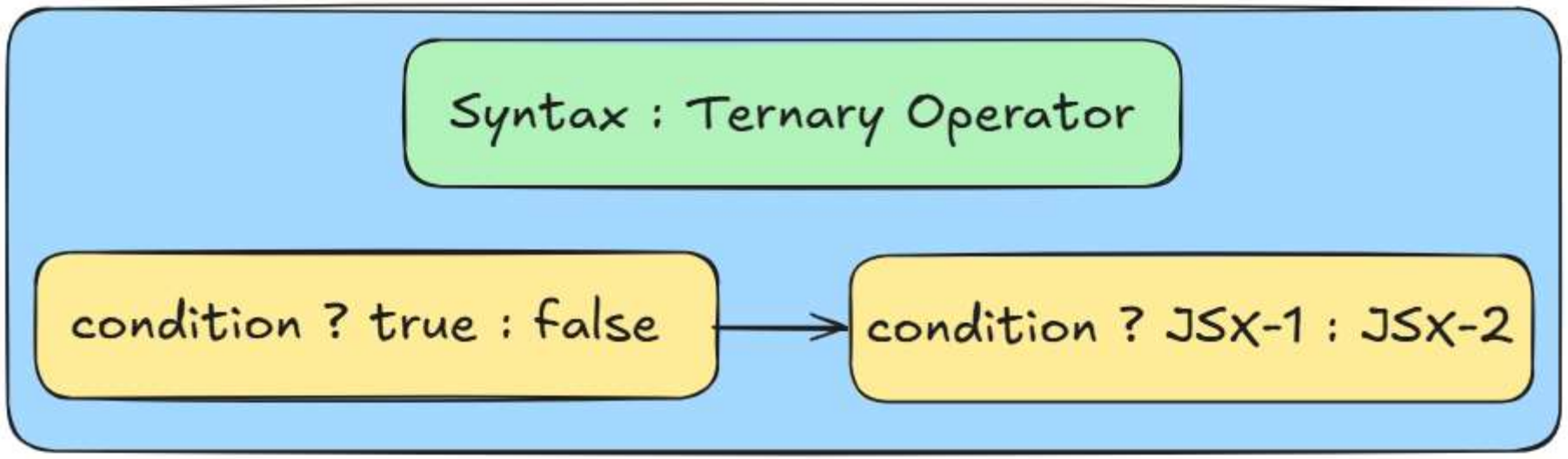
1. Conditional Rendering By Using if-else

- If-else **cannot be** used inside JSX.
- It works outside the **return()** statement of the component.



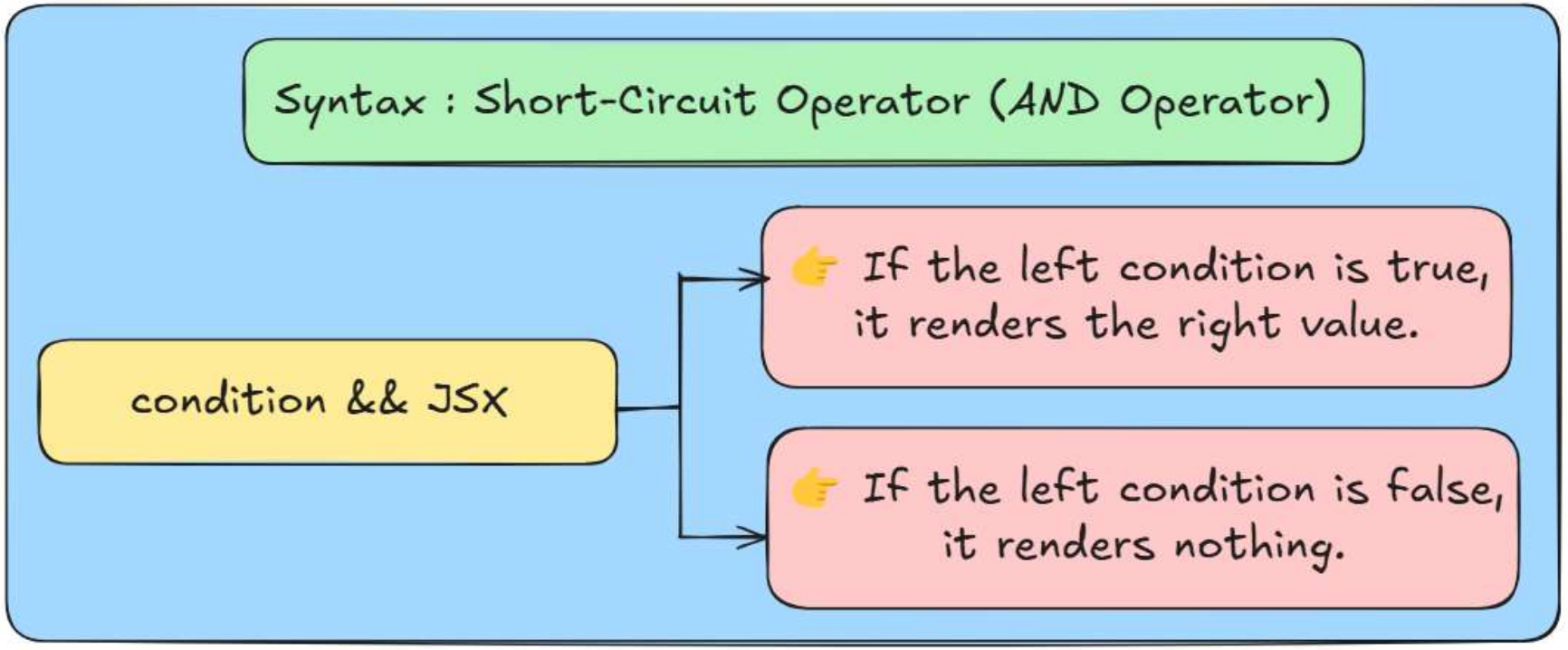
2. Conditional Rendering By Using Ternary Operator

- The ternary operator is used as a **shorter syntax** for the if-else statement.
- It works inside the **JSX** ({ }).



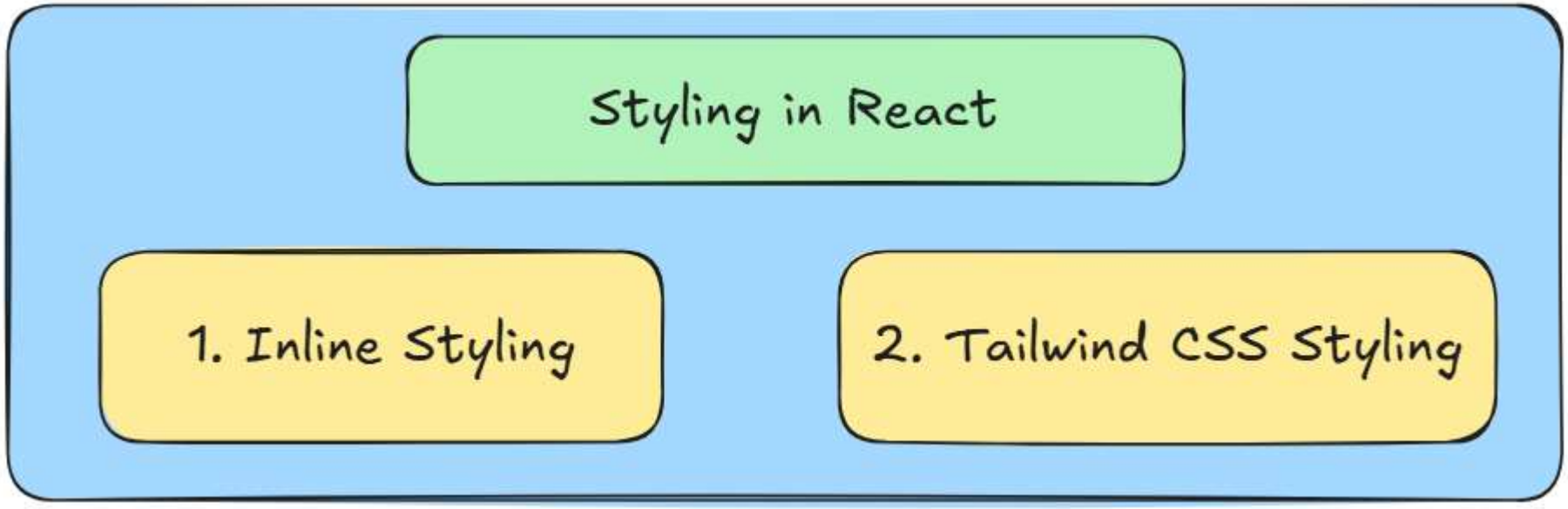
3. Conditional Rendering By Using Short-Circuit

- Short Circuit (**&&**) (**AND Operator**).
- The short-circuit operator works from **left to right**.
- It works inside the **JSX** (**{ }**).



Styling in React

- In react, you can style the components by using CSS.
- There are 2 ways you can provide the styling in react:



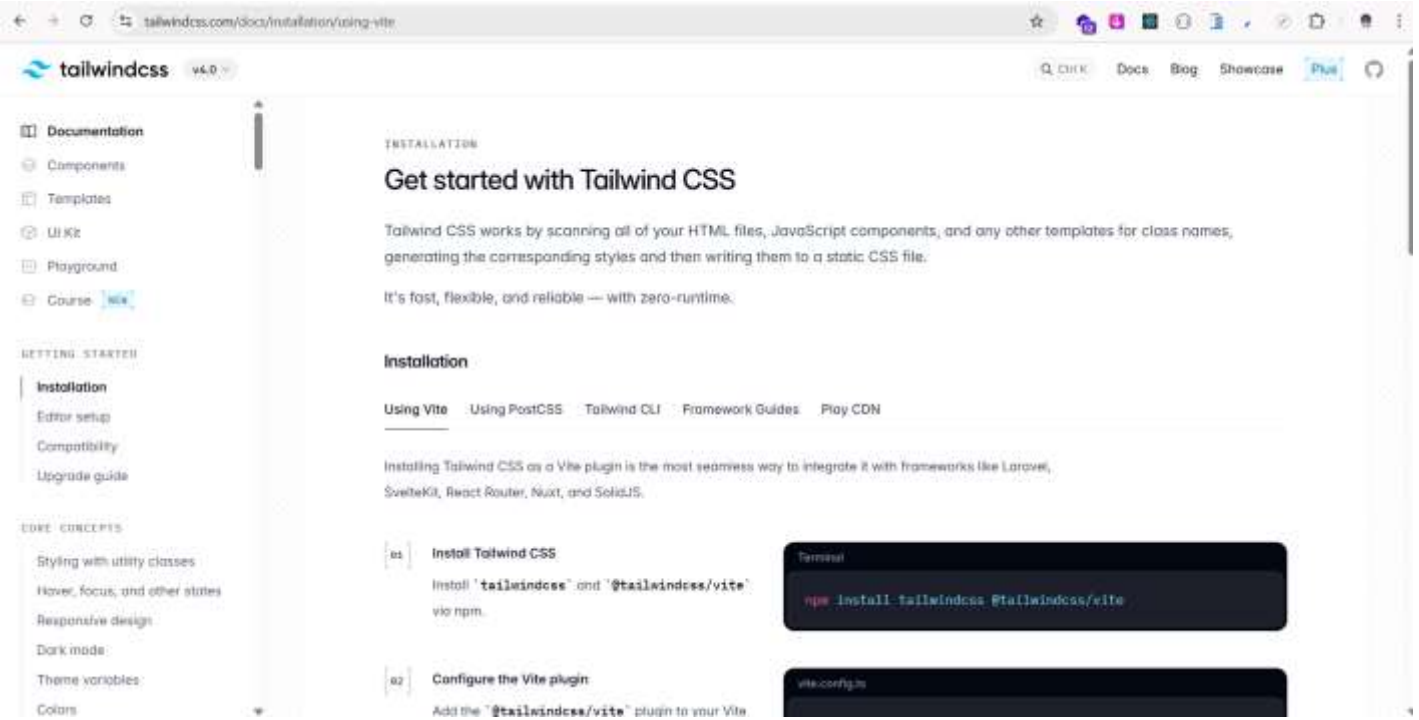
1. Inline Styling in React

- Inline styles in React are written as **Javascript Objects** inside the **style** attribute.
- Javascript object means key and value pair.
- **Key** – **propertyName** (camelCase convention)
- **Value** – **"value"** (In String)

```
1  <h1 style={{ color: "red", fontSize: "24px" }}>Inline Styling Example</h1>
```

2. Installation for Tailwind CSS for React

- First close your project (terminate the project in terminal by clicking **ctrl+c** – then type **y** and hit enter)
- Go to this website - https://tailwindcss.com/docs/installation/using-vite
- You will see this type of UI.



Step-1: Copy the first command

- Copy the first command and open new terminal - Command prompt – and paste the command and hit enter.

01

Install Tailwind CSS

Install `tailwindcss` and `@tailwindcss/vite` via npm.

Terminal

```
npm install tailwindcss @tailwindcss/vite
```

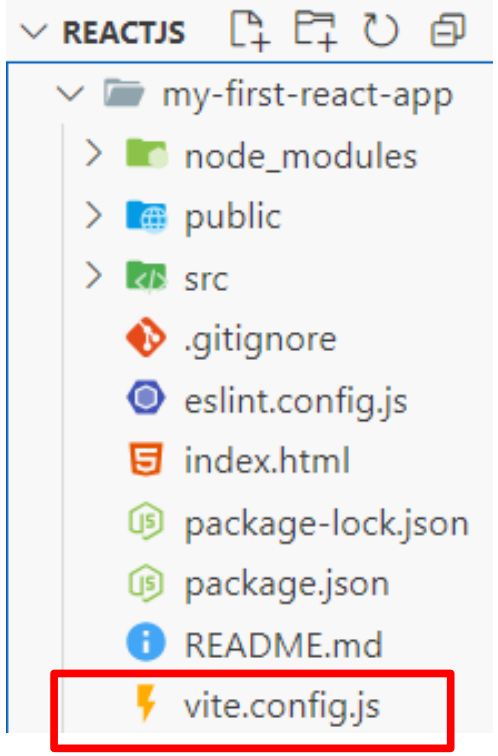
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

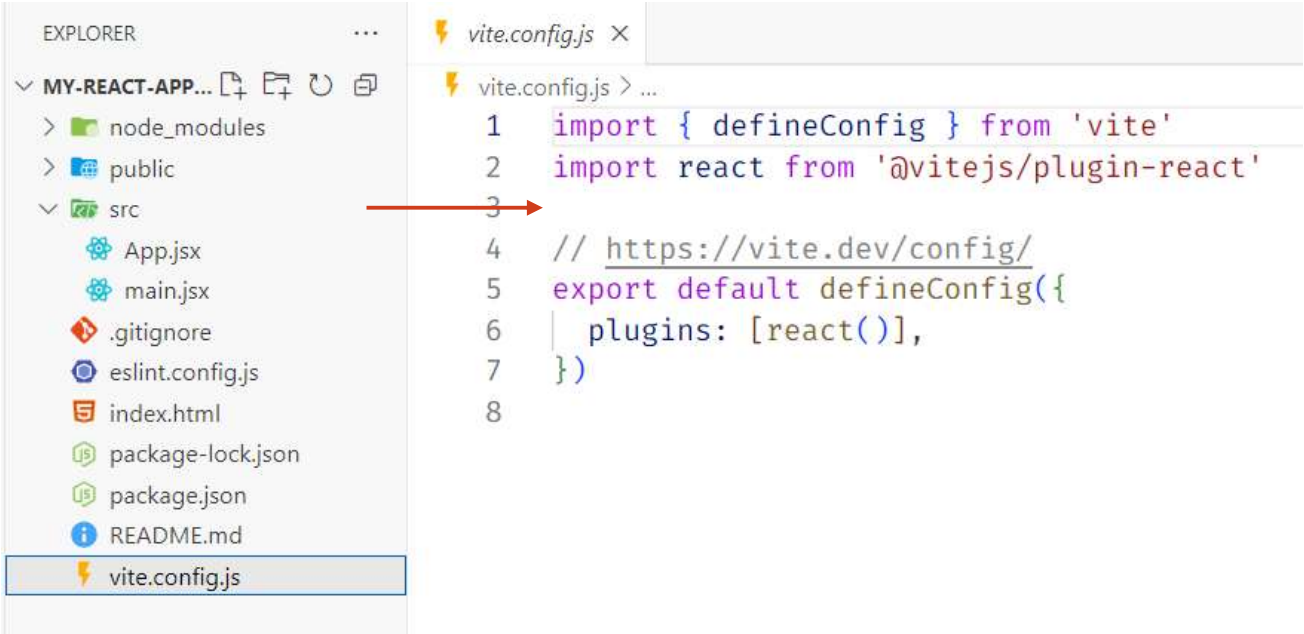
```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shridhan\Desktop\demo\my-react-app-with-tailwind>npm install tailwindcss @tailwindcss/vite
```

powershell
cmd

Step-2: Copy the 2nd import statement

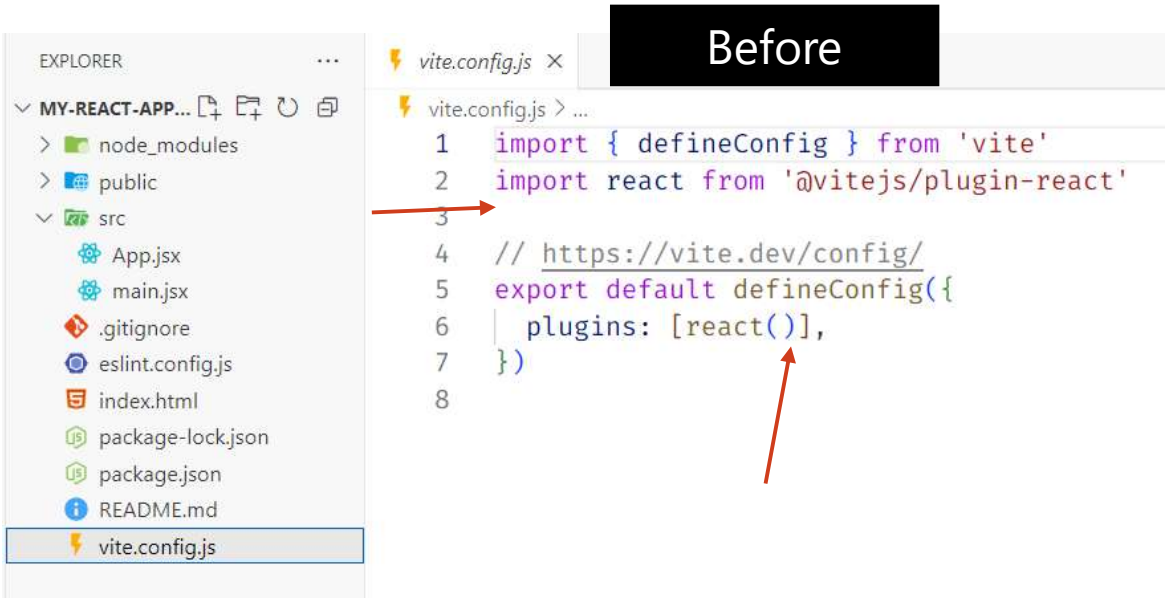
- Copy the 2nd import statement only - **import tailwindcss from '@tailwindcss/vite'**
- Then go to vs code you will see file **vite.config.js** at the last of the project.
- Open the file paste that import statement in the line no-3.

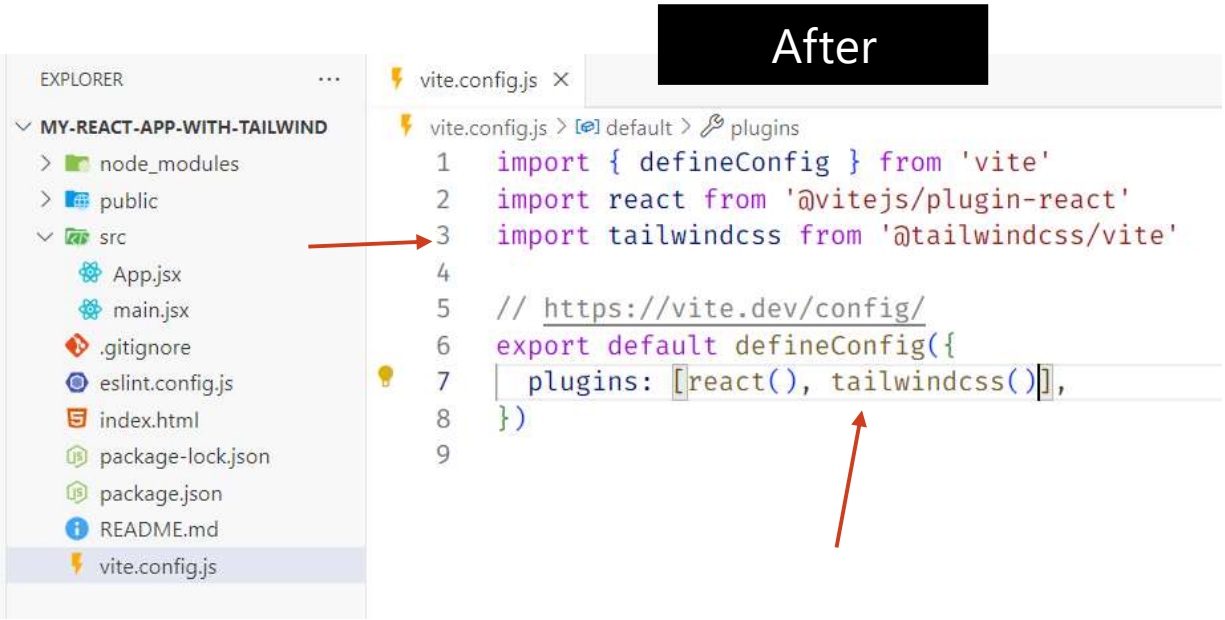




Step-3: Copy the tailwindcss()

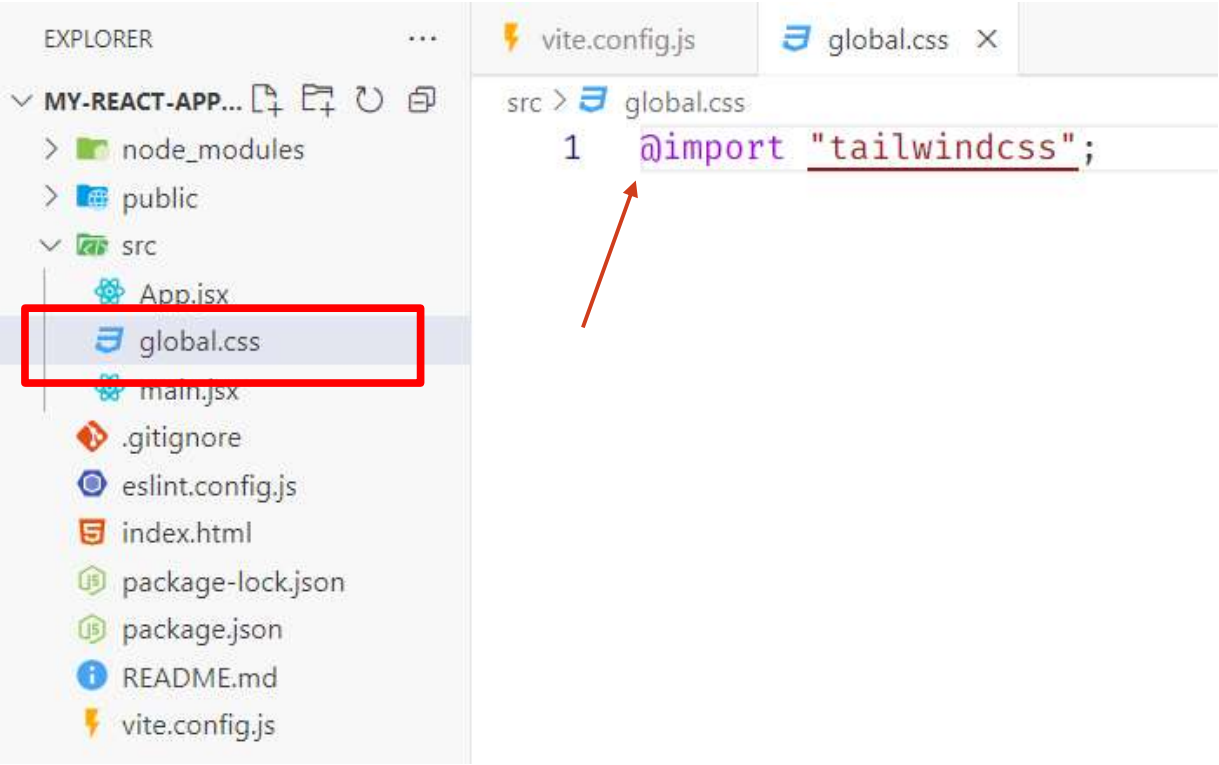
- Now copy the **tailwindcss()** from the plugins option.
- After that go to vs code, **vite.config.js** – inside you can see already react is their provide **one comma** (,) and paste that **tailwindcss()** their itself.





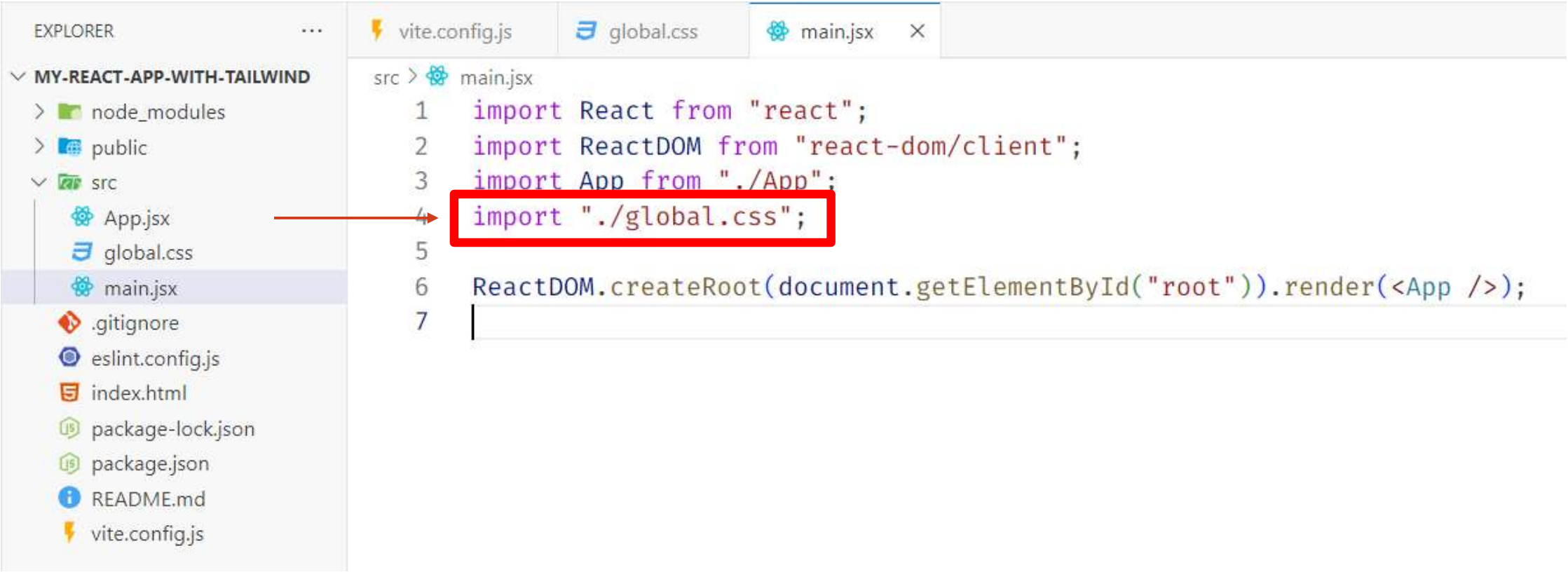
Step-4: Copy the 3rd command and paste in global.css

- Now create one file inside the **src** folder. – filename - **global.css**
- Inside that paste the 3rd command - **@import "tailwindcss";**



Step-5: Now import that global.css in main.jsx

- Now import that global.css file import inside the **main.jsx** file.
- Or you can type exact same import statement inside your code in line-4



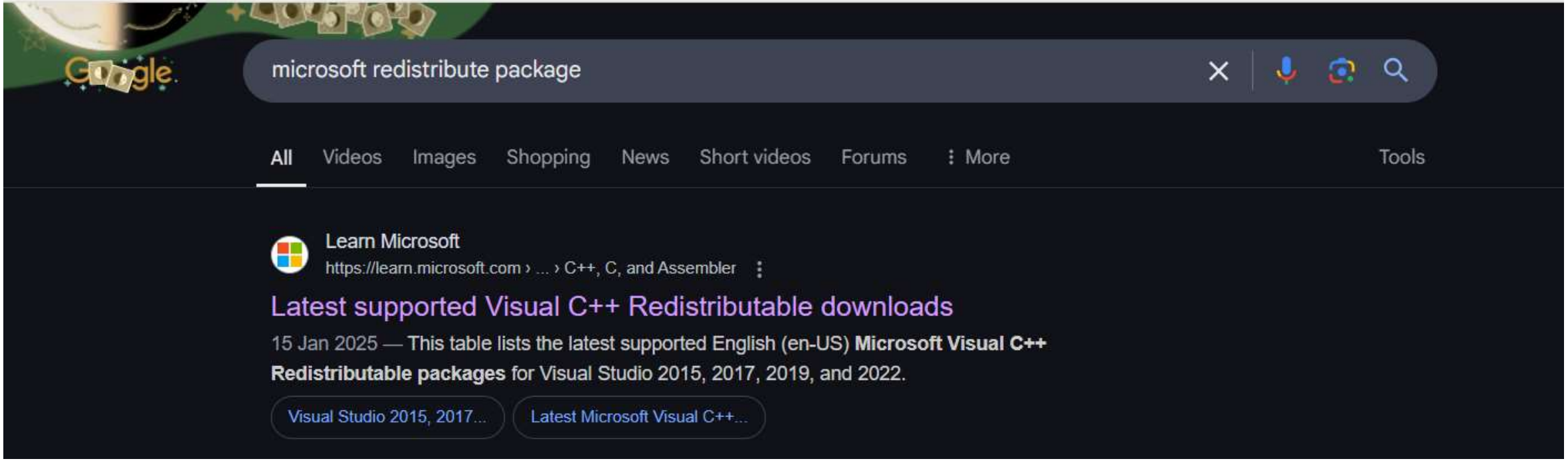
Step-6: Finally run the react project – npm run dev

- Tailwind installation done for your system.
- Now just run the react project in terminal – command prompt – **npm run dev**
- Open the link in the browser that it done.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS


32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Shridhan\Desktop\demo\my-react-app-with-tailwind>npm run dev

> my-react-app-with-tailwind@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

  VITE v6.2.2  ready in 856 ms

  → Local:   http://localhost:5174/
  → Network: use --host to expose
  → press h + enter to show help
```

For those who are getting errors

- If your are getting error like **lightningcss module not found.**
- Type **Microsoft redistribute package** in any browser and click on the first link.



For those who are getting errors

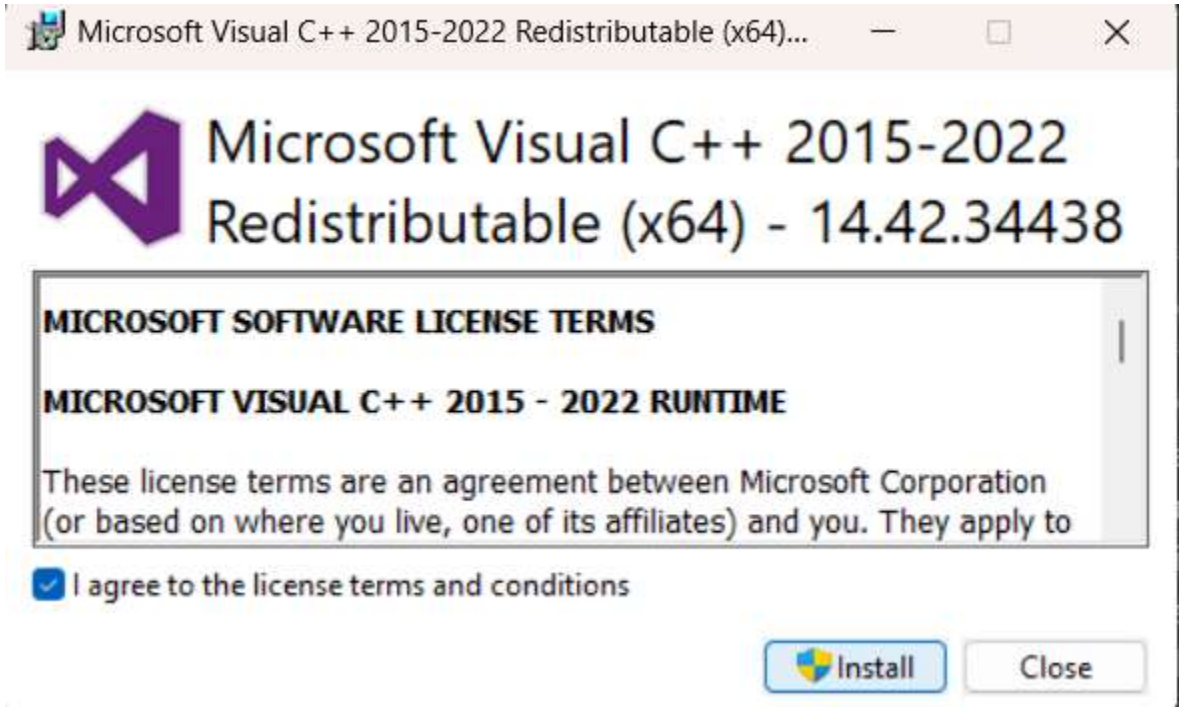
- Next thing just scroll the page you will see this type of one box.
- In the terminal, just check what it the version error coming and it may be asking like **x64** or **x86** according that just click on the link and download that package in your system.

Expand table

Architecture	Link	Notes
ARM64	https://aka.ms/vs/17/release/vc_redist.arm64.exe	Permalink for latest supported ARM64 version
X86	https://aka.ms/vs/17/release/vc_redist.x86.exe	Permalink for latest supported x86 version
X64	https://aka.ms/vs/17/release/vc_redist.x64.exe	Permalink for latest supported x64 version. The X64 Redistributable package contains both ARM64 and X64 binaries. This package makes it easy to install required Visual C++ ARM64 binaries when the X64 Redistributable is installed on an ARM64 device.

For those who are getting errors

- After downloading just check the **I agree** and click on the **install** button.
- After the completion of installation just **restart your laptop**.
- Open the vs code and run the react project now that error will not come.



React.js

DAY-4

Topics Covered:

1. Forms Handling in React
2. Key things required (4)
3. Fields to handle in forms
    i. type = text, email, password
    ii. type = radio
    iii. type = select

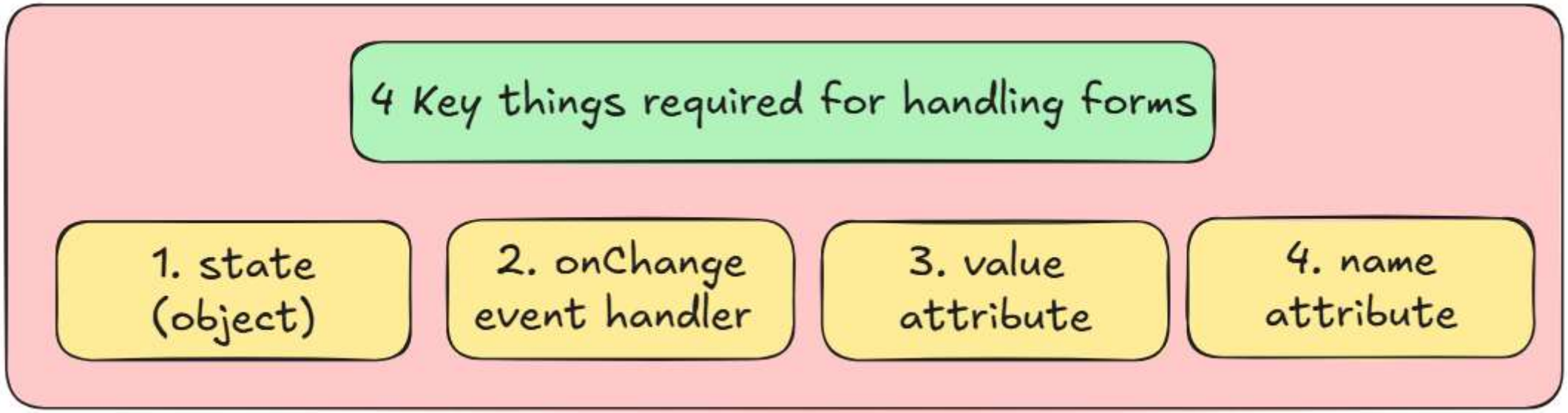
Forms Handling React

- Form handling in react means **managing the input fields** (like text, email, password, radio, select, etc) and their values when a user types something.
- **It includes:**

  1. Capturing user input
  2. Updating the state when input changes
  3. Submitting the form

Forms Handling React

- To handle forms in React properly, we need **four key** things:



Forms Handling React

1. State :

- ✓ The **state** holds the form's input values.
- ✓ It updates whenever the user types something in the input fields.

2. onChange event :

- ✓ The **onChange** event updates the state when the user types something in the input field.

3. value attribute :

- ✓ The **value** attribute binds the input field to the state.

4. name attribute :

- ✓ The **name** attribute helps identify the input field and allows us to update the correct state property.

Controlled Forms in React

- A **Controlled Form** is a form where the input field values are controlled by React state.

React controls the input.

- Value comes from useState
- On every change → (state updates)
- UI always reflects state

Un - Controlled Forms in React

Uncontrolled Form is a form where input values are handled by the DOM itself instead of React state.

Browser controls the input.

- No state required
- Uses **useRef**
- React does not track input value

UseRef

useRef

useRef is a hook that lets you store a value or reference a DOM element without triggering a re-render.

1. Store a value **without re-rendering** the component
2. Access or control a **DOM element directly**

React.js

DAY-5

Topics Covered:

1. useEffect()
2. Context API

useEffect()

- The **useEffect()** is the React Hook.
- It helps us to handle side-effect .
- (Side-Effect) -Any operation that happens outside of the normal React rendering flow

It is mainly used for:

1. Fetching data from an API
2. Dom Manpulation

What Happens If we don't use useEffect() Hook

- If we put API calls **directly inside the component**, they will **run on every re-render**.

1. **Infinite API requests**

2. **Unwanted side effects**

Side Eeffect A **side effect** is any operation performed inside a component that affects something outside its scope or interacts with external systems, other than returning JSX for rendering.

useEffect() Hook

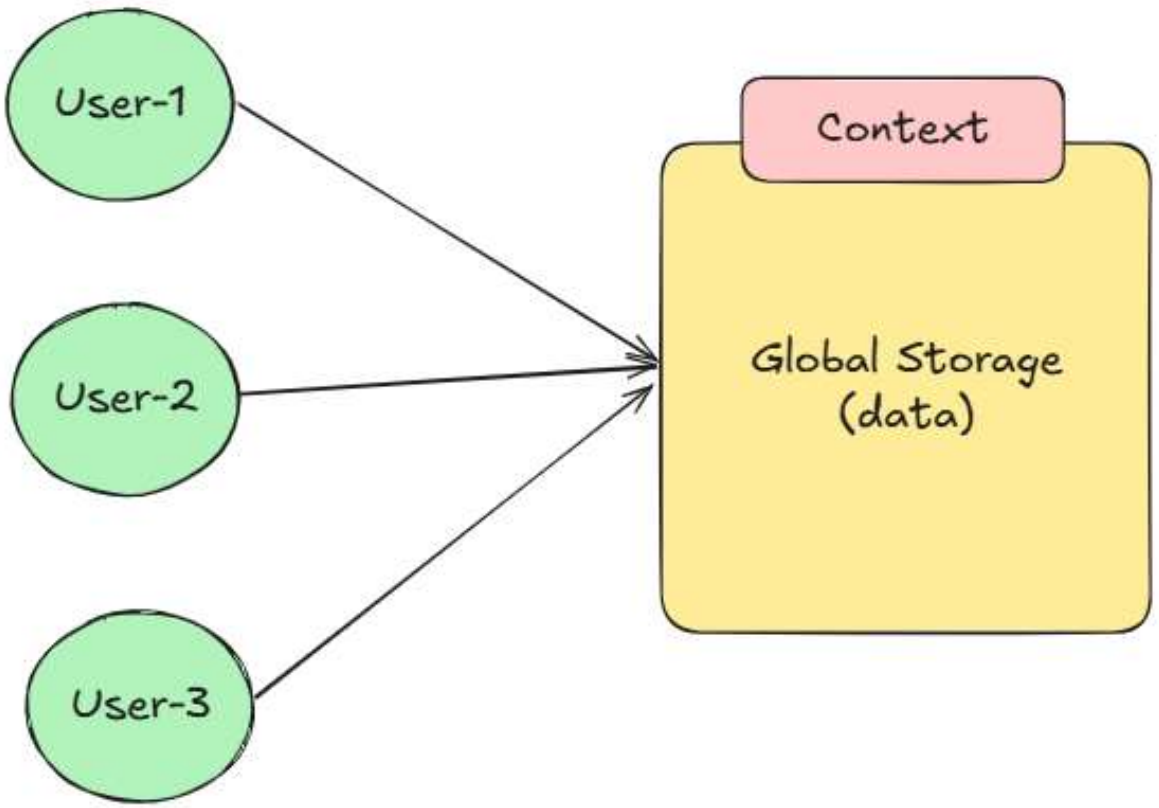
Basic Syntax of useEffect()

```
import { useEffect } from "react";

useEffect(() => {
      // Your code here (runs after render)
}, [dependency_array]);
```

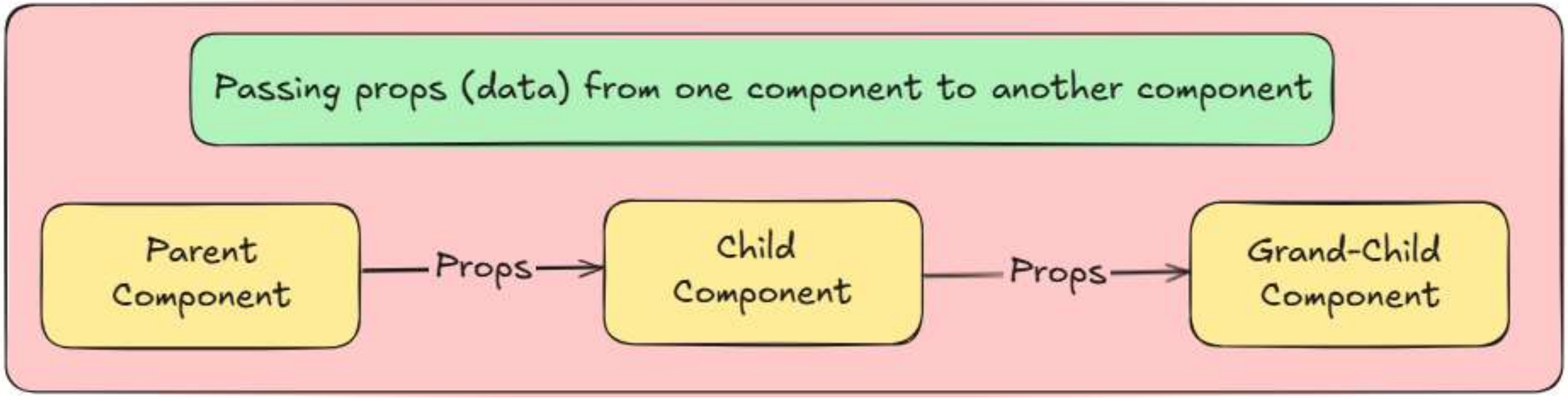
1. No dependency array – Runs every render
2. Empty ([ ]) array – Runs only once
3. Dependency Array with values – Runs when dependencies change

Context API in React

- **Context API** is a way to **share data** (state) between multiple components without passing props manually at every level.
- Think of it like a **global storage** for your app.

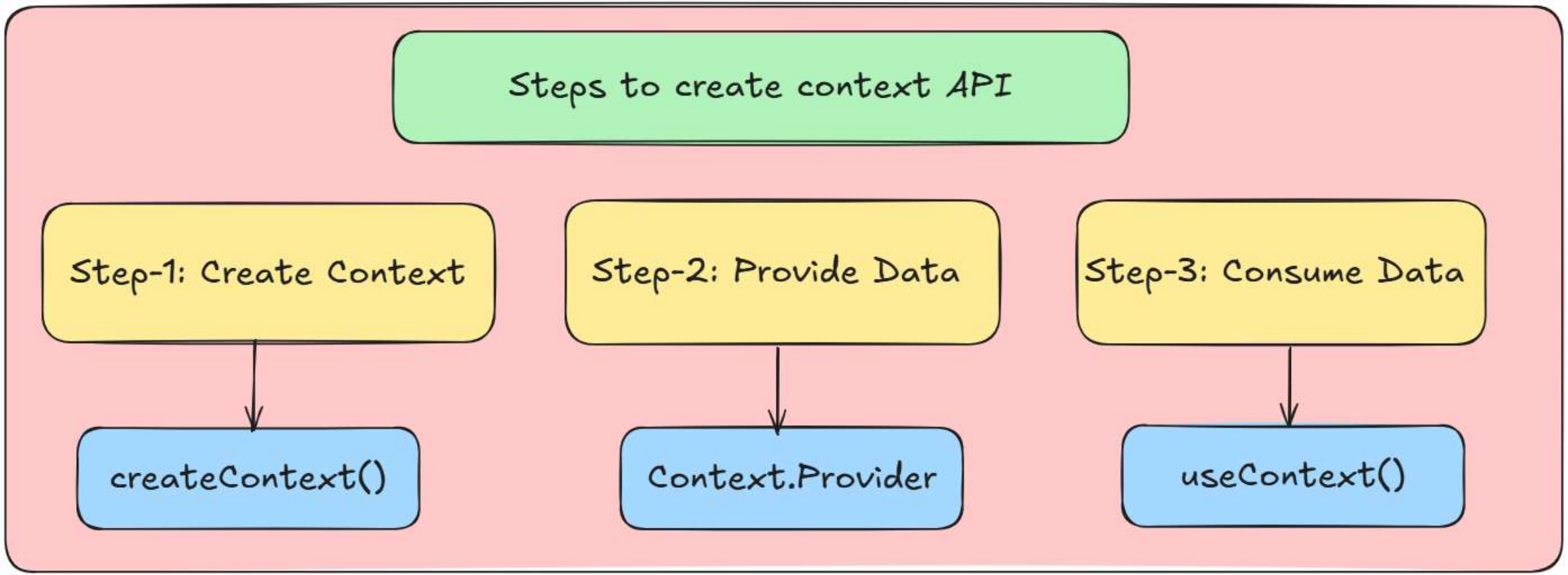


Why Do We Need Context API?



- Normally, we pass data from **parent** → **child** → **grandchild** using **props**.
- But in large applications, this can be difficult (called "prop drilling").
- **Context API** helps avoid this by **providing** data **directly** to any component that needs it.

Steps to Create Context API

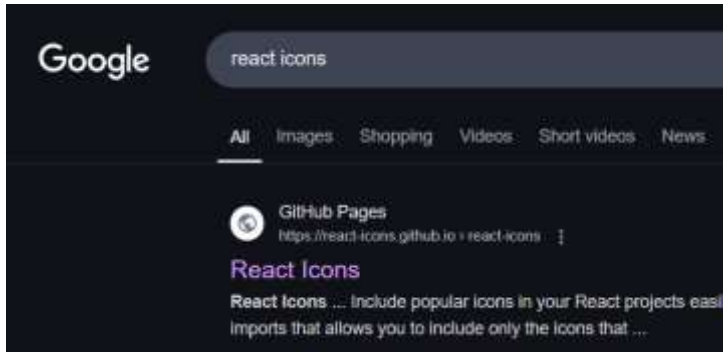


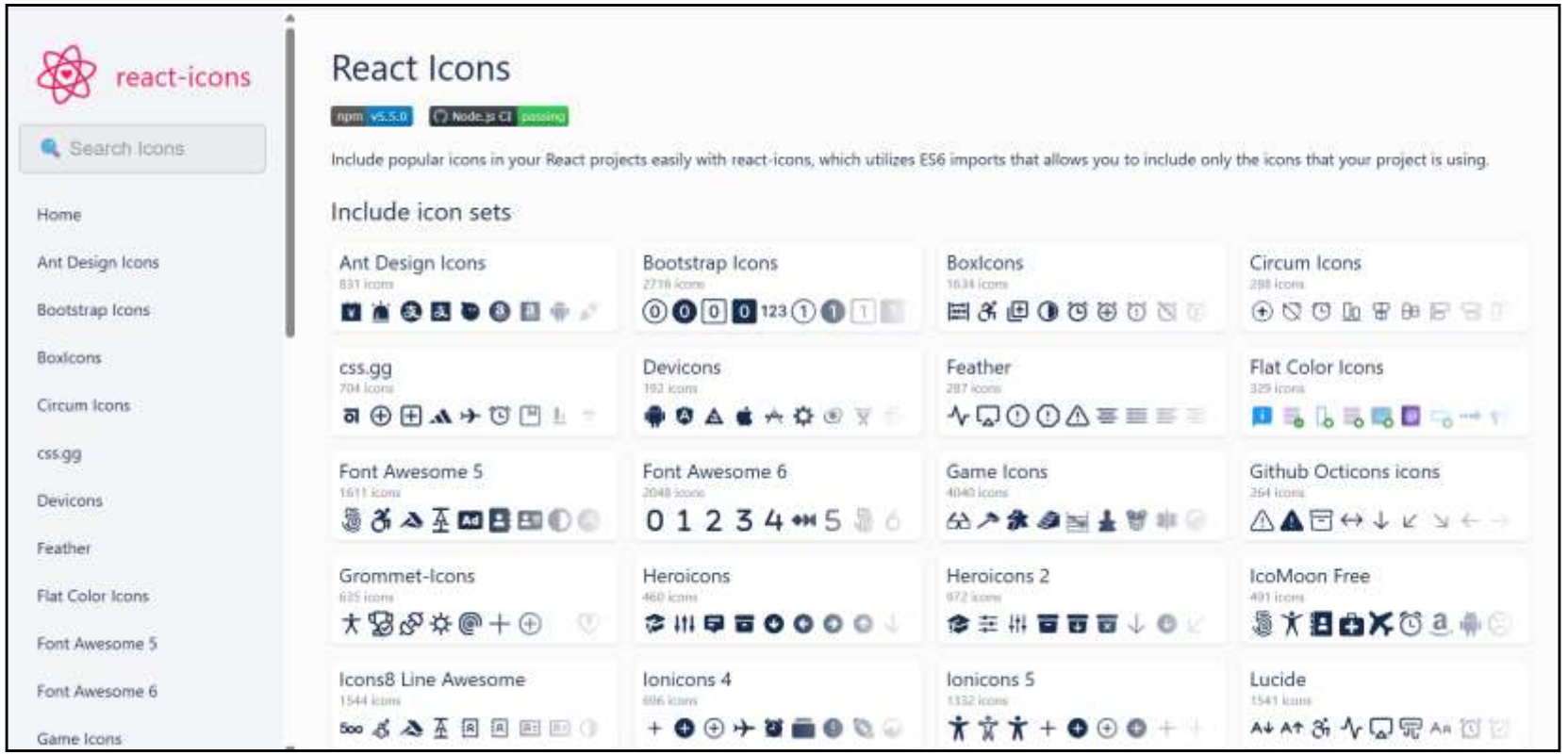
Steps to Create Context API

1. **Create Context**: const MyContext = createContext();
2. **Provide Data**: <MyContext.Provider value={data}>{children}</MyContext.Provider>
3. **Consume Data**: const data = useContext(MyContext);

How to install react icons

- Open terminal – new terminal – command prompt – **npm install react-icons**
- Type react icons on any browser click on the first link – **React Icons**



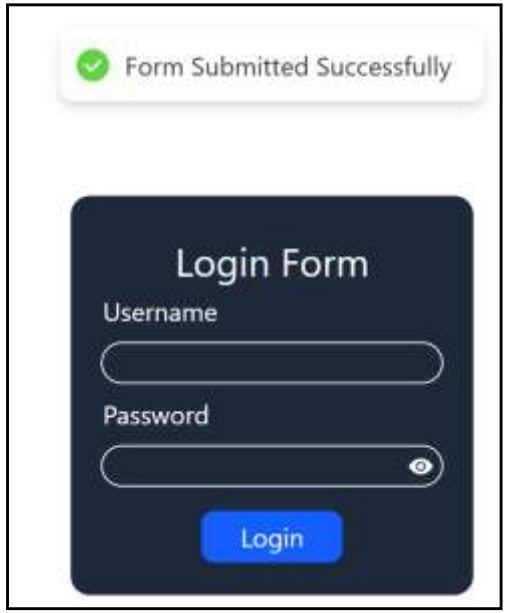


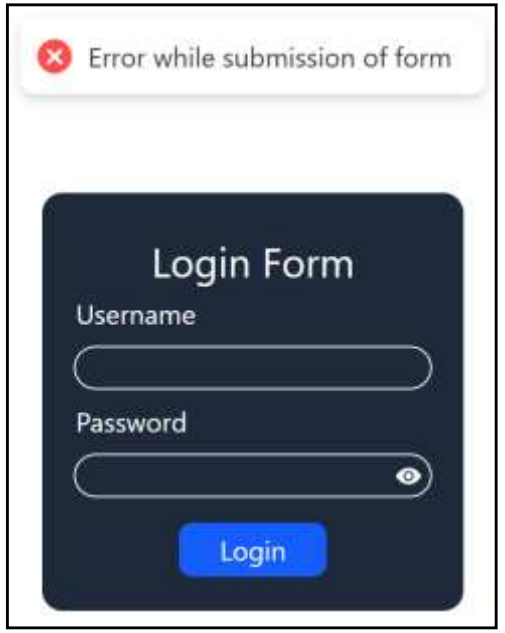
How to use toast notifications

- Open terminal – new terminal – command prompt – **npm install react-hot-toast**
- For success notification – **toast.success("msg")**
- For error notification – **toast.error("msg")**
- Don't forget to import the **<Toaster/>** component inside the **main.jsx** component.
- Without this toast notification will not work.

main.jsx

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./global.css";
import { Toaster } from "react-hot-toast";

ReactDOM.createRoot(document.getElementById("root")).render(
  <>
    <App />
    <Toaster />
  </>
);
```





React.js

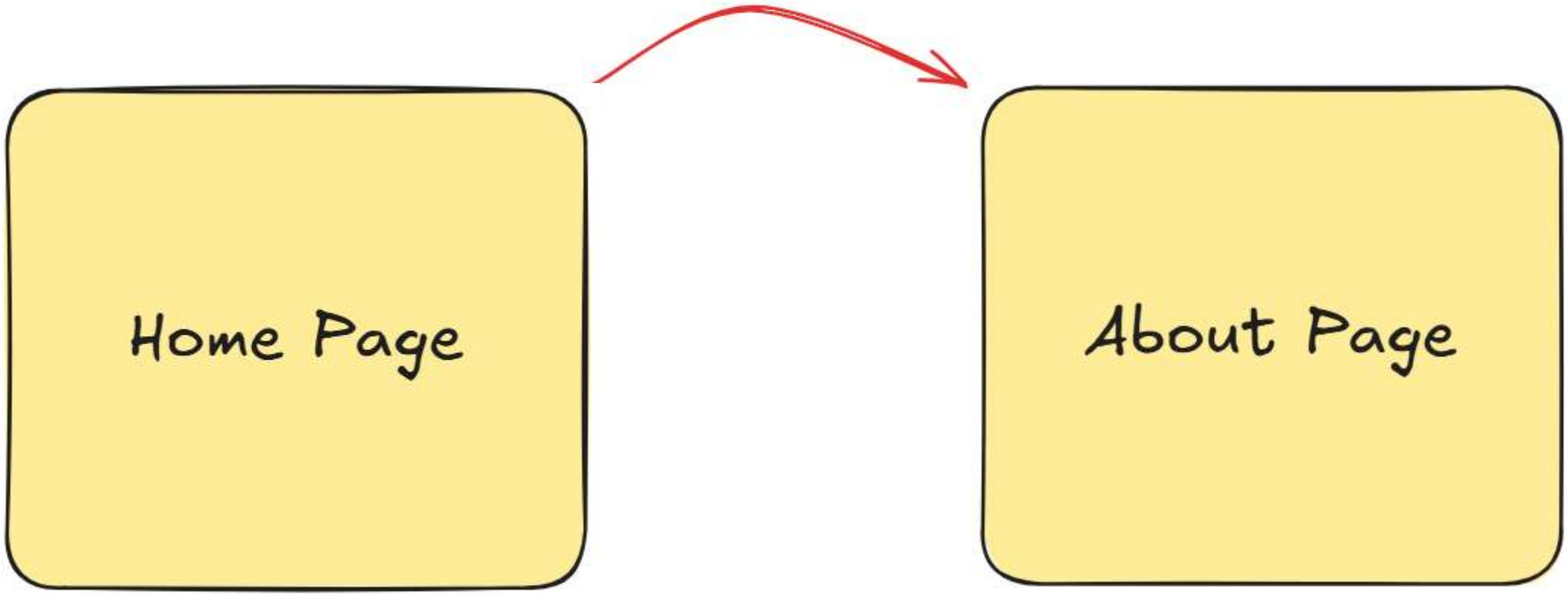
DAY-6

Topics Covered:

1. Routing in React js

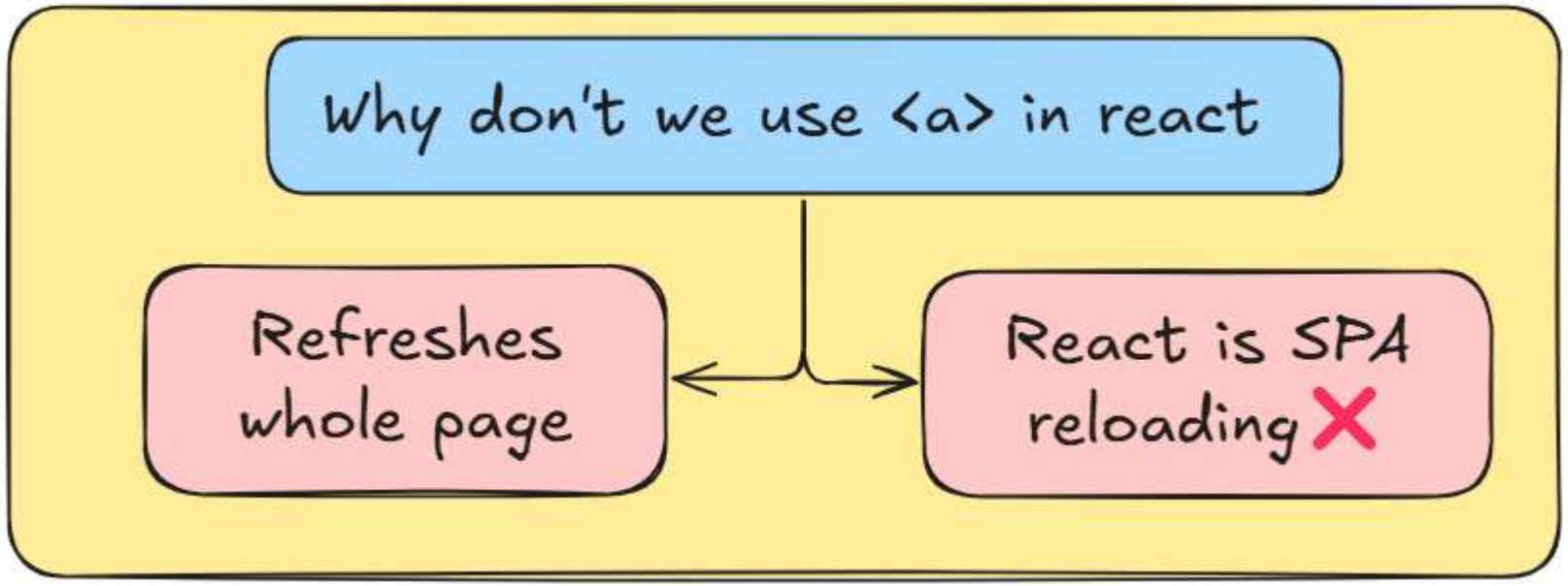
Routing in React

- **Routing** is the process of navigating from one page to another page in web application.



What is Page Routing?

- **Page Routing** refers to switching between different pages in a website.
- Traditionally, clicking a link reloads the entire page, but in **Single Page Applications** (SPAs).
- So in react, routing changes only parts of the page without a full reload.



What is React Routing?

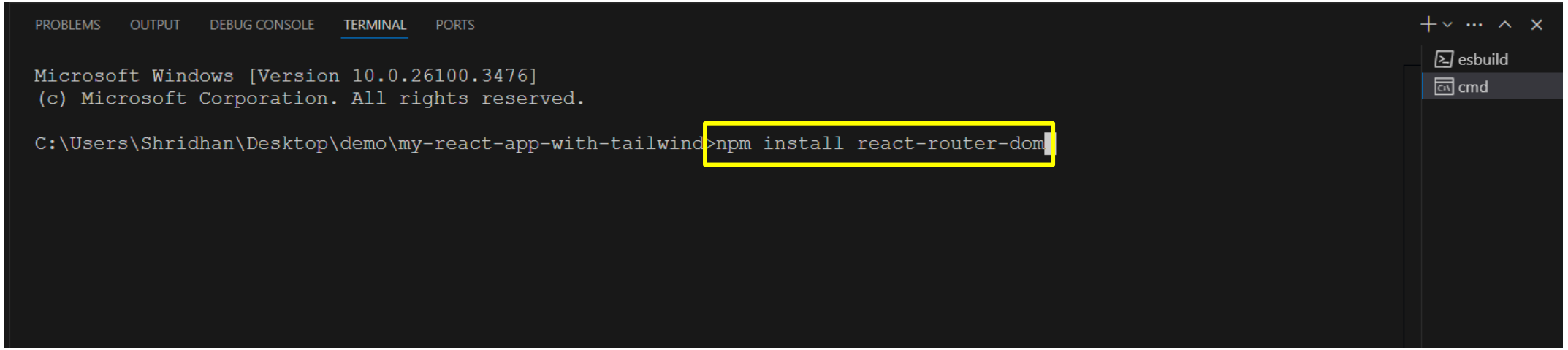
- **React Routing** allows you to create multiple pages in react app without reloading the page.
- We can achieve react routing with the help of **React-Router-Dom** in react.

Q. What is React Router DOM?

- React Router DOM is a **library** for handling navigation in React applications.
- It provides tools like
  1. createBrowserRouter()
  2. <Link>
  3. <NavLink>

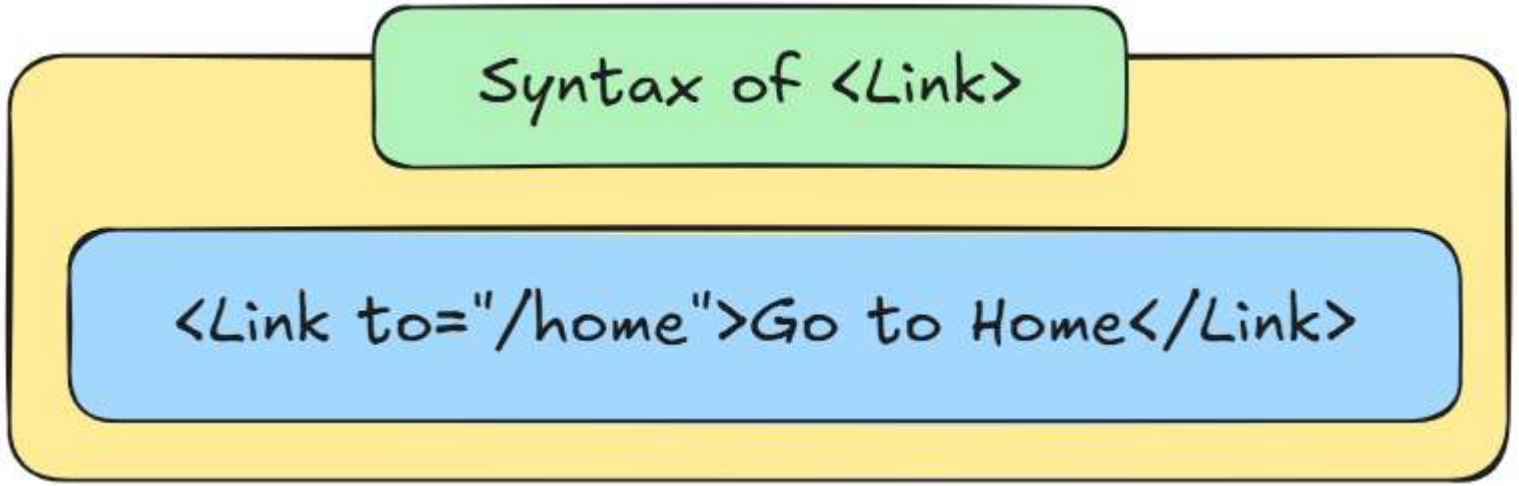
Installation of react-router-dom

- Open terminal – new terminal – command prompt – **npm install react-router-dom**
- And hit enter



What is <Link> ?

- **<Link>** is used to navigate between pages without refreshing the browser.



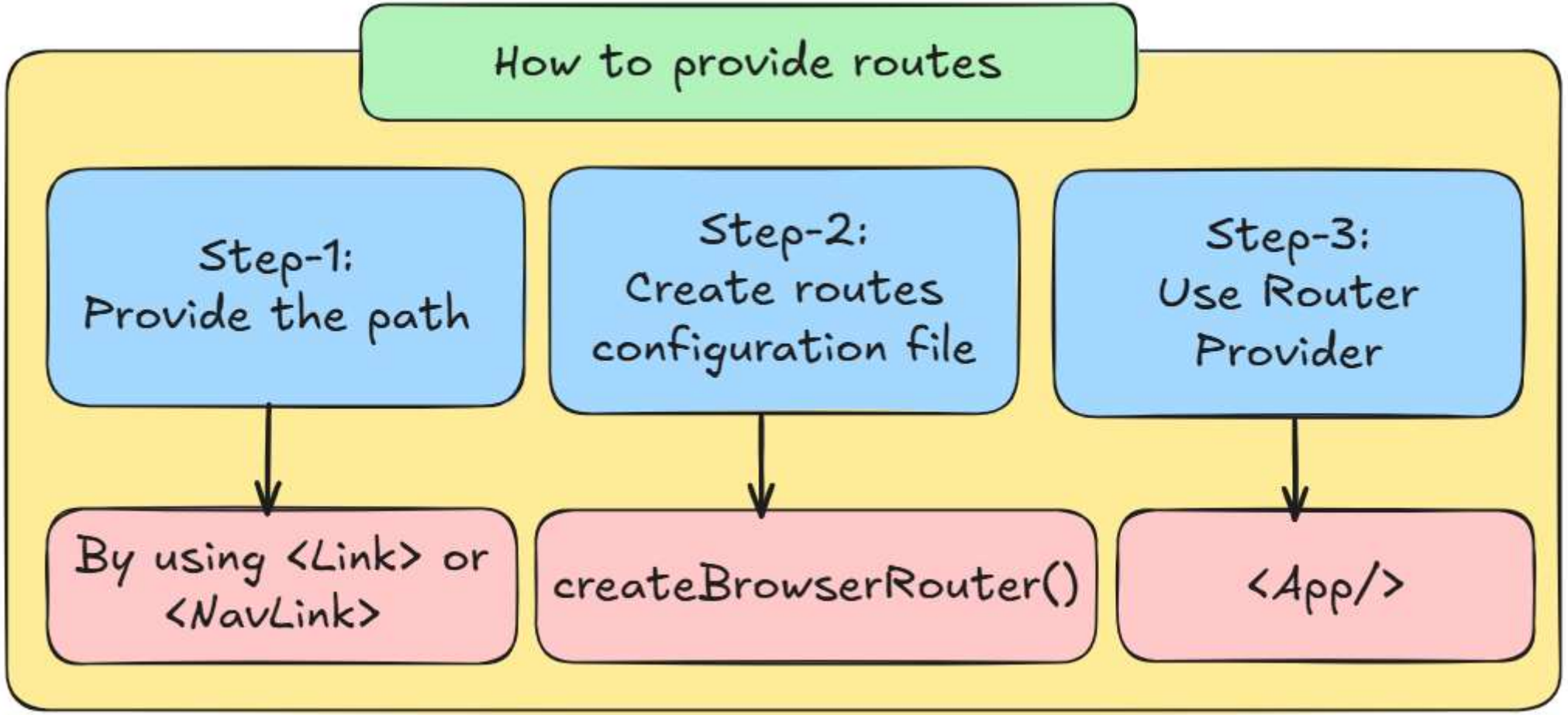
What is <NavLink> ?

- **<NavLink>** is similar to **<Link>**, but it adds an **active class** automatically when the link is active.

Syntax of <NavLink>

```
<NavLink to="/home" activeClassName="active">
        Go to Home
</NavLink>
```

Steps to provide the routes in react



What is createBrowserRouter()?

- The **createBrowserRouter()** is a newer way to define routes in structured way.
- This method will accept one **array** of an **objects**.

Syntax of createBrowserRouter()

```
import { createBrowserRouter } from "react-router-dom";

const router = createBrowserRouter([
    { path: "/", element: <Home /> },
    { path: "/about", element: <About /> },
]);
```

What is <Outlet/> ?

- **<Outlet/>** is a placeholder that renders child components inside a parent layout component.

Syntax of <Outlet/>

```
let Layout = () => {
    return (
        <div>
            <h1>Website Header</h1>
            <Outlet />
              {/* Child components will be displayed here */}
        </div>
    );
}
```

What is useNavigate()?

- The **useNavigate()** is hook in react.
- The useNavigate() is used to navigate programmatically between pages in React Router.
- Instead of using <NavLink> and <Link>, you can navigate using useNavigate() hook.

Syntax of useNavigate()

```
import { useNavigate } from "react-router-dom";

const navigate = useNavigate();
navigate("/home"); // Redirects to /home
```